



**JARAMOGI OGINGA ODINGA UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATICS AND INNOVATIVE SYSTEMS
BACHELOR OF SCIENCE (INFORMATION AND COMMUNICATION
TECHNOLOGY)**

**PROJECT
ICB 1404**

BRIAN KIMONGE (I231/0050/2019)
VINCENT OWUOR (I231/1073/2019)

**TITLE:
FACIAL RECOGNITION ATTENDANCE MANAGEMENT SYSTEM**

SUPERVISED BY:
PROF. SOLOMON OGARA
AND
PROF. RODRIQUES

A Final Project Document Submitted to the Board of School of Informatics and Innovative Systems in Partial Fulfillment of the Requirements for the Award for Bachelor of Science in Information and Communication Technology at Jaramogi Oginga Odinga University of Science and Technology

APRIL 2023

DECLARATION AND APPROVAL

Declaration

This project is our original work and has not been done for an award of a diploma or degree in any other university or institution.

Signature: Date:
VINCENT OWUOR
I231/1073/2019

Signature: Date:
BRIAN KIMONGE
I231/0050/2019

Approval

Supervisor

This proposal has been submitted with our approval as the university supervisors.

Signature: Date:
Prof: Rodriguez
School of Information and Innovative Systems
Jaramogi Oginga Odinga University of Science and Technology

Signature: Date:
Prof. Solomon Ogara
School of Information and Innovative Systems
Jaramogi Oginga Odinga University of Science and Technology

ABSTRACT

Facial recognition attendance management system is a web based computerized system that uses facial recognition technology to accurately and efficiently record and manage students' attendance. The system works by capturing a facial image of the student using a camera, analyzing the image using facial recognition algorithms to match it with the student's stored image in the database, and recording the attendance data in real-time. The system offers numerous benefits over traditional attendance management systems, including increased accuracy, reduced costs, and improved efficiency. It also eliminates the need for manual record-keeping, reduces the chances of errors and attendance forgery, and provides real-time reports on attendance data.

TABLE OF CONTENT

Table of Contents

DECLARATION AND APPROVAL.....	2
ABSTRACT.....	3
TABLE OF CONTENT	4
List of Tables	6
List of Figures	7
Acronyms/Symbols/Abbreviations	9
1. CHAPTER ONE: INTRODUCTION.....	10
1.1. Background Information.....	10
1.2. Statement of the Problem	11
1.3. Objectives.....	11
1.3.1. GENERAL OBJECTIVE	11
1.3.2. SPECIFIC OBJECTIVES.....	11
1.4. Significance of the Study.....	11
1.5. Scope.....	11
1.6. Definition of Terms	12
2. CHAPTER TWO: LITERATURE REVIEW	13
2.1. Introduction	13
2.2. Implementation	13
2.3. Research.....	13
2.4. Conceptual Framework.....	14
3. CHAPTER THREE: METHODOLOGY.....	15
4. CHAPTER FOUR: ANALYSIS.....	16
4.1. What is analysis?.....	16
4.2. System Requirements:.....	16
4.2.1. Non-functional requirement.....	16
4.2.2. Functional requirement	17
5. CHAPTER FIVE: DESIGN	18
5.1. What is design?.....	18
5.2. Project Diagrams.....	18
5.2.1. Use-case Diagram.....	18

5.2.2.	Flowcharts Diagrams.....	20
5.2.3.	Sequence Diagrams.....	22
5.2.4.	Context Diagram	26
5.2.5.	Entity Relation Diagram (ERD)	27
6.	IMPLIMENTATION.	29
7.	WORK PLAN.....	94
8.	BUDGET	95
9.	REFERENCE.....	96

List of Tables

Table 1 : Work Plan	94
Table 2: Project Budget.	95

List of Figures

Figure 5-1 Use-Case Diagram.....	19
Figure 5-2 Admin Flowchart	20
Figure 5-3 Student Flowchart	21
Figure 5-4 Staff Flowchart.....	22
Figure 5-5 Staff Sequence diagram.....	23
Figure 5-6 Student sequence diagram.....	24
Figure 5-7 Admin sequence diagram	25
Figure 5-8 Data flow diagram Level 0.....	26
Figure 5-9 Data Flow diagram Level 1	27
Figure 5-10 Entity Relation Diagram.....	28
Figure 6- 1 index page for route	81
Figure 6- 2 student registration page	82
Figure 6- 3 student login page	82
Figure 6- 4 student home page.....	83
Figure 6- 5 student view attendance	83
Figure 6- 6 student viewing and attendance taking	84
Figure 6- 7 staff registration	85
Figure 6- 8 staff login	85
Figure 6- 9 staff home page	86
Figure 6- 10 staff attendance.....	86
Figure 6- 11 staff creating classes	87
Figure 6- 12 staff view courses	87
Figure 6- 13 staff unit selection	88
Figure 6- 14 admin login page.....	89
Figure 6- 15 admin home page	89
Figure 6- 16 admin view all students	90
Figure 6- 17 admin add student.....	90
Figure 6- 18 admin view staffs	91
Figure 6- 19 admin adding staffs.....	91
Figure 6- 20 admin view all courses.....	92
Figure 6- 21 admin course registration.....	92
Figure 6- 22 admin view all units	93
Figure 6- 23 admin unit registration	93

Acronyms/Symbols/Abbreviations

All symbols, abbreviations and acronyms in the thesis should be listed on this page and their full interpretations, and units where appropriate given.

SDLC - System Development Life Cycle.

FRAMS – Facial Recognition Attendance Management System.

OPENCV – Open Computer Vision.

UML – Unified Modeling Language.

1. CHAPTER ONE: INTRODUCTION

1.1. Background Information

A biometric system is essentially a pattern recognition system that operates by acquiring biometric data from an individual, extracting a feature vector from the acquired data, comparing this feature vector from the database feature vector. Person authentication and verification has always been an attractive goal in computer vision. Authentication systems based on human characteristics such as face, finger, iris and voice are known as Biometrics systems. Face recognition is a method of identifying or verifying the identity of an individual using their face. It is a technology capable of matching a human face from a digital image or a video frame against a database of faces, typically employed to authenticate users through ID verification services, works by pinpointing and measuring facial features from a given image.

Facial recognition was first introduced and started development as a form of computer application in the 1960s. Since then, this facial recognition system has seen wider uses in recent times on smartphones and in other forms of technology, such as robotics as it computerizes facial recognition involving the measurement of a human's physiological characteristics, thus categorized as biometrics. Although the accuracy of facial recognition systems as a biometric technology is lower than iris recognition and fingerprint recognition, it is widely adopted due to its contactless process. Facial recognition systems have been deployed in advanced human-computer interaction, video surveillance and automatic indexing of images.

In most educational institutions, the attendance is taken manually. It is not only time consuming, but it is also insecure and unreliable and it can be lost. Some institutions are using cards for attendance, while this will be difficult for teachers to keep track of the large number of students because by using cards, a student can help the other students or his/her friend to their card even the other student may be absent or come late in class, so it is not reliable.

To overcome these problems, we have developed a better system which is Web based; it is fully responsive where a user can use in mobile or different computer systems. In this system, records are kept safe and secure, the attendance information of particular or all students of particular class can be accessed easily, and without time consuming, the report is generated automatically.

Attendance is very important in every student. A single absent make a big difference in performance in the school. Most students of high school and higher learning institutions are prone to absence from classes claiming that the class is boring. Others due to laziness fail to attend classes, having preference of going to computer shops or playing games while some students cannot refuse the prospect of a friend asking them out during class period. With this, the authority would communicate the parents on the attendance level of the student.

1.2. Statement of the Problem

Attendance is taken with student face before joining or entering the lecture room. The daily attendance of students is taken automatically by scanning the student's face with a camera. The attendance report will be generated automatically without time consuming which is reliable and there will be not any mistakes. It is fully responsive and flexible; the school dean with the attendance portal can access it and produce it when needed by the examination office or any other authority who is in charge of it.

1.3. Objectives

1.3.1. GENERAL OBJECTIVE

The main objective of the project is to come up with a system that will be used to automate the manual management of class attendance records, the application is web based thus will be access anywhere provided you are connected to the internet.

1.3.2. SPECIFIC OBJECTIVES

- i. Study the previous system and mark changes to the problems that were previously not considered.
- ii. To design system
- iii. To develop and implement the system.
- iv. To test and implement the system

1.4. Significance of the Study

- (a) Conducting a research and requirement analysis for the making of the project overall view based on models and system designing.
- (b) Research findings and requirement analysis will be beneficial in the development of the project properly suited for the targeted consumers both being the staffs and the students.

1.5. Scope

Facial recognition is becoming more widely applicable in our today's society making a major progress in the field of security. It is a very effective tool that can help law enforcers to recognize criminals and software companies are leveraging the technology to help users access the technology. This technology can be further developed to be used in other avenues such as ATMs, accessing confidential files, or other sensitive materials.

This project serves as an addition to the already present foundation on facial detection and recognition. This project also covers web development and database management with a user-friendly UI. Using this system any school and organization can replace their traditional way of maintaining attendance of the consumer and can also generate their availability(presence) report throughout the month.

1.6. Definition of Terms

Attendance

Biometric system

Facial recognition

Management system

2. CHAPTER TWO: LITERATURE REVIEW

2.1. Introduction

As introduced in the first chapter, biometric system is a pattern recognition system that operates by acquiring biometric data from an individual through the extraction of feature vector and comparing the feature from the database feature vector.

2.2. Implementation

According to an article published in www.govtech.com, there are evidence of the use of biometrics dating back to Babylon in 500BC. The Babylonians used clay tablets to capture and store fingerprint during business transactions. By the mid-1800s, the industrial revolution sparked rapid city growth, and a standard form of identifying the general public and criminal was necessary. Some police adopted the Bertillon system (anthropometric), invented in France, which recorded the limb and body measurements on index cards. With no standards in places led to frequent errors hence fingerprint became the method of choice in the late 1800s till today.

Face recognition was first used in Super Bowl Tampa in 2001. It captured an image of each of 100,000 fans via the security camera and checked electronically against mug shots from Tampa police.

In 2003, a Federal Government Coordination was started by the National Science and Technical Council in which established an official subcommittee on biometrics, and a year later the Department of Defense Implemented the Automated Biometric Identification System (ABIS) to help track and identify national security threat.

2.3. Research

"Facial Recognition" edited by Ashutosh Tiwari and Swarup Kumar Majumder: This book provides an overview of facial recognition technology, including its history, current state of development, and applications in various fields. It includes articles from experts in the field of computer vision and facial recognition.

"Handbook of Face Recognition" edited by Stan Z. Li and Anil K. Jain: This comprehensive reference book covers all aspects of face recognition, including theory, algorithms, applications, and evaluation methods. It includes contributions from leading researchers in the field and is considered a definitive resource on facial recognition.

"Advances in Face Detection and Facial Image Analysis" edited by Feng Li and Stan Z. Li: This book focuses on the latest advances in face detection and facial image analysis using machine learning and deep learning algorithms. It includes articles on face detection, face recognition, facial expression analysis, and other related topics.

"Face Recognition Across the Imaging Spectrum" edited by Sameer Singh and Umapada Pal: This book covers face recognition technology across various imaging spectrums, including visible light, infrared, and thermal imaging. It includes articles on the latest advances in face recognition technology and its applications in different fields, such as security, healthcare, and entertainment.

"Facial Analysis from Continuous Video with Applications to Human-Computer Interface" by Fernando De la Torre and Jeffrey F. Cohn: This book focuses on facial analysis using continuous video, including facial expression recognition and facial landmark detection. It includes articles on the latest techniques and applications of facial analysis in human-computer interface design.

2.4. Conceptual Framework

Facial recognition technology for security systems: This conceptual framework focuses on the use of facial recognition technology in security systems to identify individuals and monitor their movements in various establishments, such as airports, border crossings, and public places. This helps as it provides relevant information of every individual based on the already data present in the system.

Facial recognition in healthcare: This framework explores the potential of facial recognition technology in healthcare, including its use in patient identification, monitoring of vital signs, and detecting early signs of diseases based on facial changes.

Facial recognition and emotional recognition: This framework explore the relationship between facial recognition and emotional recognition, including how facial expressions can be used to detect emotions and how emotions can be used to improve facial recognition technology.

Facial recognition and bias: This framework examine the potential for bias in facial recognition technology, including issues related to accuracy, privacy, and discrimination based on race, gender, or other factors.

Facial recognition and social media: This framework explore the use of facial recognition technology in social media, including its potential for identifying individuals in photos and videos, detecting fake profiles, and enhancing user experience through personalized content.

3. CHAPTER THREE: METHODOLOGY

We will use **waterfall model** in the designing of the system as it takes the fundamental role of activities of specification, development, validation and evolution and represents them as a separate process phase in this system development life cycle (SDLC). These fundamental activities are: requirement analysis and definition, system and software design, implementation, testing and maintenance.

Requirements Analysis: In this phase, we have functional and Non-Functional Requirements for the facial recognition attendance management system are collected and documented. We shall look in details on chapter 4.

Design: At this phase, we create system design. We designed the system in a way that it will supports the facial recognition technology and meets the requirements. We shall discuss in details on chapter 5.

Implementation: At this phase, we develop the system and test it. This includes programming, testing, and debugging the code. We will test system to ensure that it works as expected.

Testing: At this phase, the system is verified to ensure that it meets the requirements. This includes testing the system and verifying that it works as expected.

Maintenance: At this phase, we will do system maintenance and updated. This includes fixing bugs, updating the system to meet new requirements, and providing ongoing support for the system.

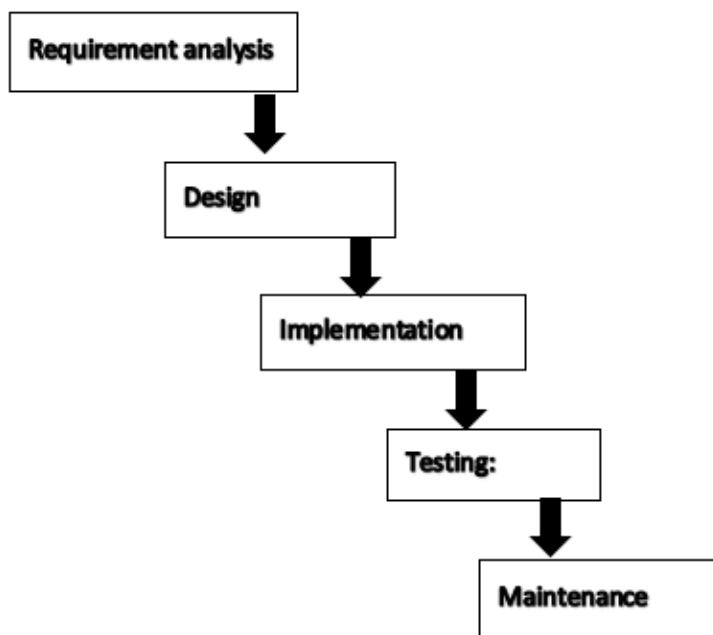


Figure3 1 Waterfall diagram

4. CHAPTER FOUR: ANALYSIS

4.1. What is analysis?

Analysis consists of activities that enable an individual to understand and specify what the new system should accomplish. It helps describe in detail what a system must do to satisfy the need or solve the problem.

Analysis of this project was done in order to come up with specific requirements that would be used in steering the project to its goals and objectives specified above in the first chapter. Through requirement analysis, we designed some Unified Model Language (UML) so as to provide a structural and logical representation of how the project might come to be started till its final documentation.

4.2. System Requirements:

4.2.1. Non-functional requirement

Image processing: The system will need to be able to process images of faces and identify specific features to be able to match them against a database of known faces. Libraries such as OpenCV and PIL based on python can be used to perform face detection, alignment, and recognition.

Face recognition algorithm: The system will require a face recognition algorithm to identify the face in the image and compare it to the database of known faces. Popular algorithms include Facenet, OpenFace and face_recognition.

Database: The system will need a database to store images of known faces and attendance records. The database should be able to efficiently store and retrieve large amounts of data. Sqlite is an example of the database software that would be used to store data.

User interface: The system will require a user interface for users to interact with the system. This will be a web-based application running on flask, a python base server for hosting and running simple website.

Real-time processing: The system should be able to process images in real-time to allow for attendance tracking in real-time.

Security: The system should be secure and protect the privacy of individuals. This includes ensuring that the system is not vulnerable to attacks, encrypting data, and following appropriate security protocols.

Accuracy: The system should be accurate in identifying faces and tracking attendance. This includes accounting for variations in lighting, facial expressions, and other factors that can affect accuracy.

Integration: The system should be able to integrate with other systems such as HR systems and payroll systems.

4.2.2. Functional requirement

Admin

- Manages the information of both student and staff through:
 - a) Inserting information
 - b) Updating information
 - c) Deleting information

Student

- Registers their information to the system.
- Log in to the system
- Take classes offered in accordance to their year of study.
- Facial features are captured for attendance marking.
- Gets personal attendance information.

Staff

- Registers their information to the system.
- Log in to the system
- Select units.
- Create classes for the student.
- Gets class attendance from the system.

5. CHAPTER FIVE: DESIGN

5.1. What is design?

Design is an activity that enable an individual to describe in detail how the system will be implemented to suite and provide the needed solution. It describes how the system is supposed to work and specifies all the component of the solution and how they work together in a system.

System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It involves developing a detailed plan and creating diagrams, models, and documentation to describe the system's structure, behavior, and functionality.

UML (Unified Modeling Language) is a popular tool for system design and modeling. It provides a standard notation for creating diagrams and models that describe the system's structure, behavior, and interactions. UML includes several types of diagrams, such as use case diagrams, class diagrams, sequence diagrams, and activity diagrams, which can be used to represent different aspects of the system. An example of UML modeling for system design could be creating a class diagram for a movie booking system that shows the classes, attributes, and methods involved in the system.

5.2. Project Diagrams

5.2.1. Use-case Diagram

In this diagram, there are three primary actors: the student, staff, and administrator. The student attends class and has their attendance recorded through the facial recognition system. The staff records attendance using the system and can also view attendance records for their classes. The administrator is responsible for managing attendance records and accessing attendance data.

The system's use cases are grouped into two categories: attendance-related and management-related. The attendance-related use cases include "Attend Class" and "Record Attendance," while the management-related use cases include "View Attendance Records," "Access Attendance Data," "Manage Attendance Records," and "Manage Attendance Data."

Registering students: This use case would allow administrators to add new students to the system and store their personal details, including their facial features.

Recording attendance: This use case would allow teachers to use the facial recognition system to record attendance for each class, and mark absentees if necessary.

Managing attendance records: This use case would allow administrators to view and manage attendance records, including generating reports for specific time periods or students.

Troubleshooting system errors: This use case would allow administrators to diagnose and resolve any errors or issues that may arise with the facial recognition system.

Configuring system settings: This use case would allow administrators to configure the system settings, including adjusting the sensitivity of the facial recognition software or setting up automatic email notifications for absentees.

Providing access to students and teachers: This use case would allow authorized students and teachers to access the system and view their attendance records.

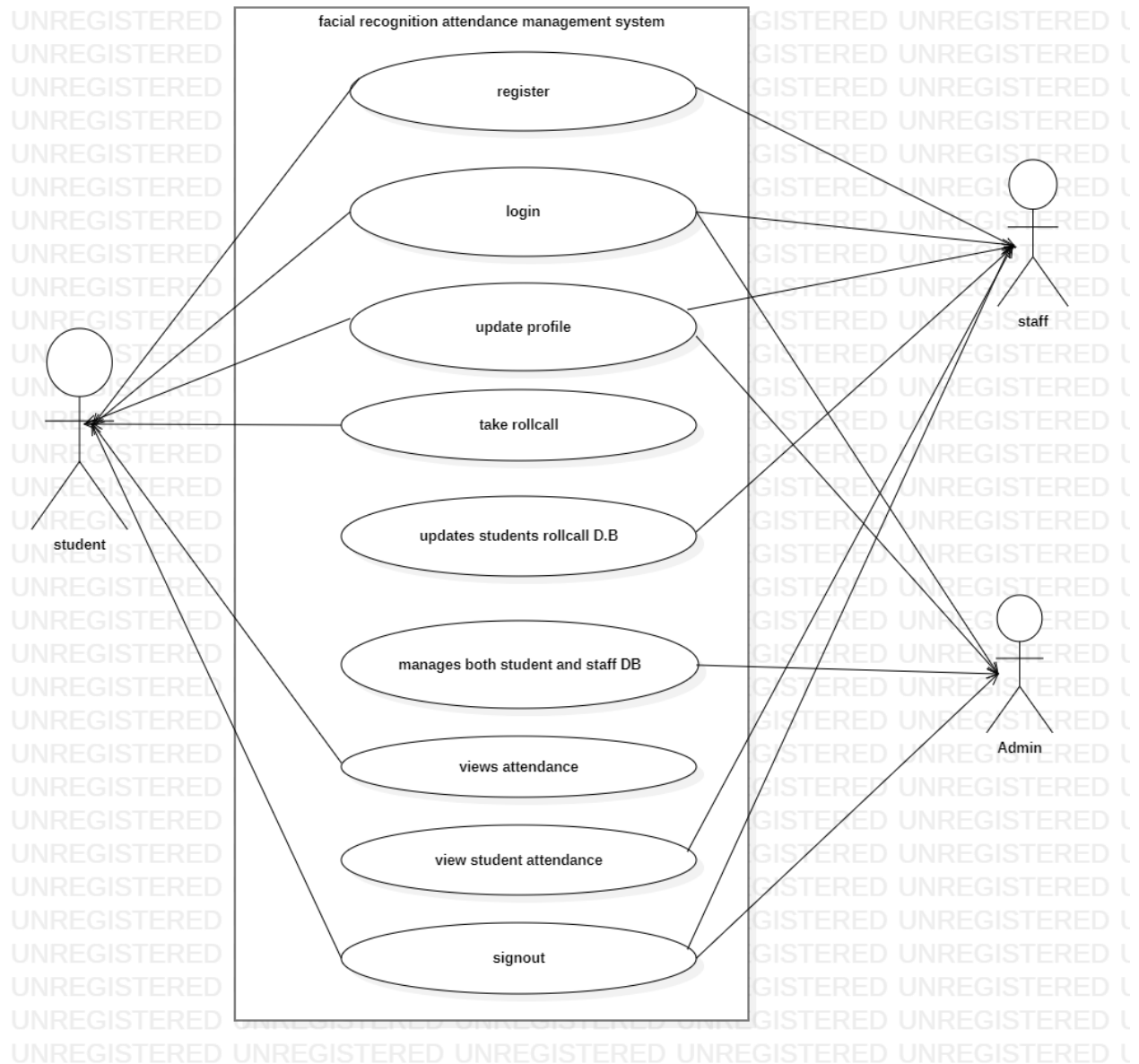


Figure 5-1 Use-Case Diagram

5.2.2. Flowcharts Diagrams

This provides the logical representation of how the system will work. With detailed information and decisions, it provides path in which the different systems based on their uniquely different functionalities would operate as a one major system known as FRAMS. This sub-systems are; admin, student and staff.

5.2.2.1. Admin flowchart

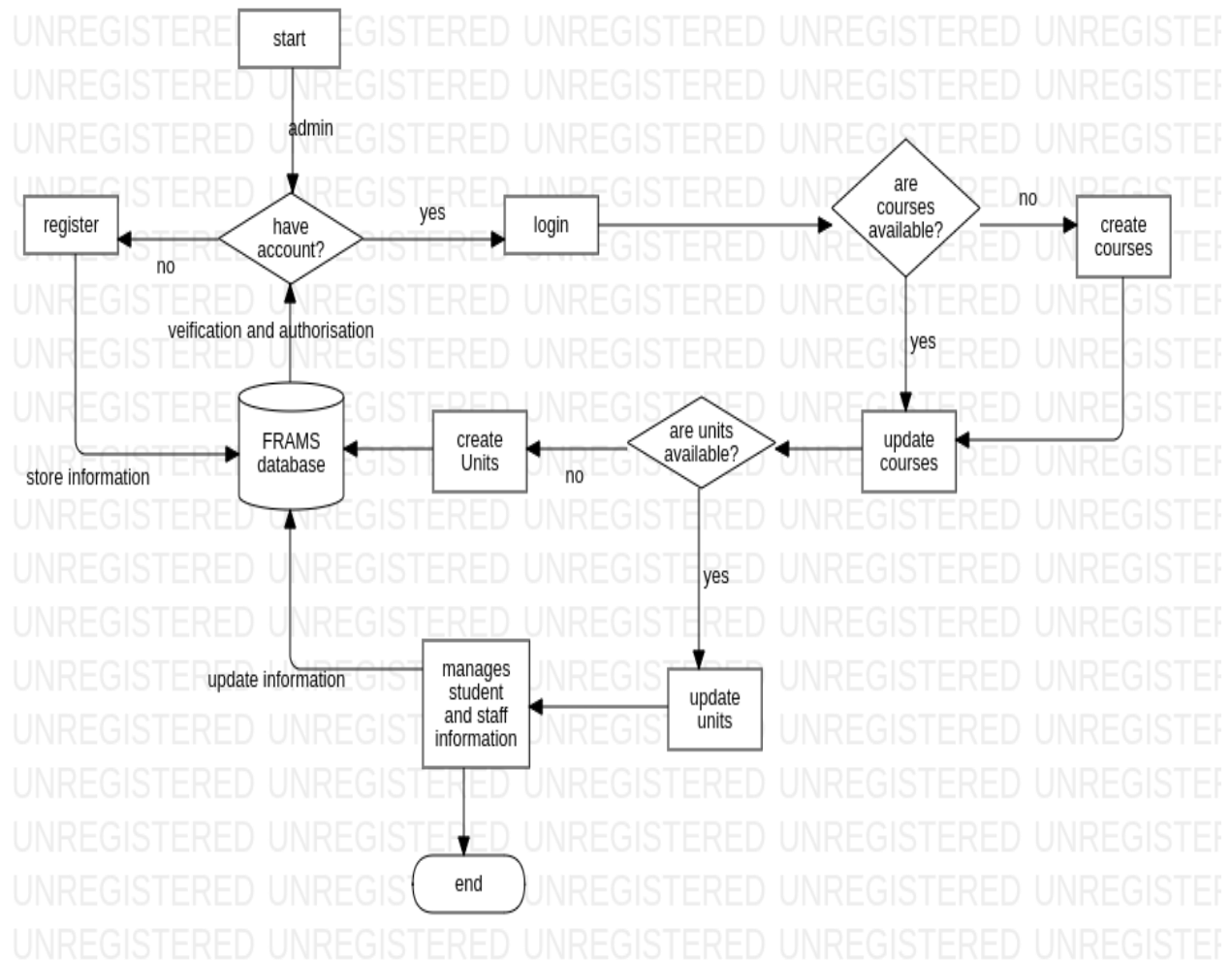


Figure 5-2 Admin Flowchart

5.2.2.2. Student flowchart

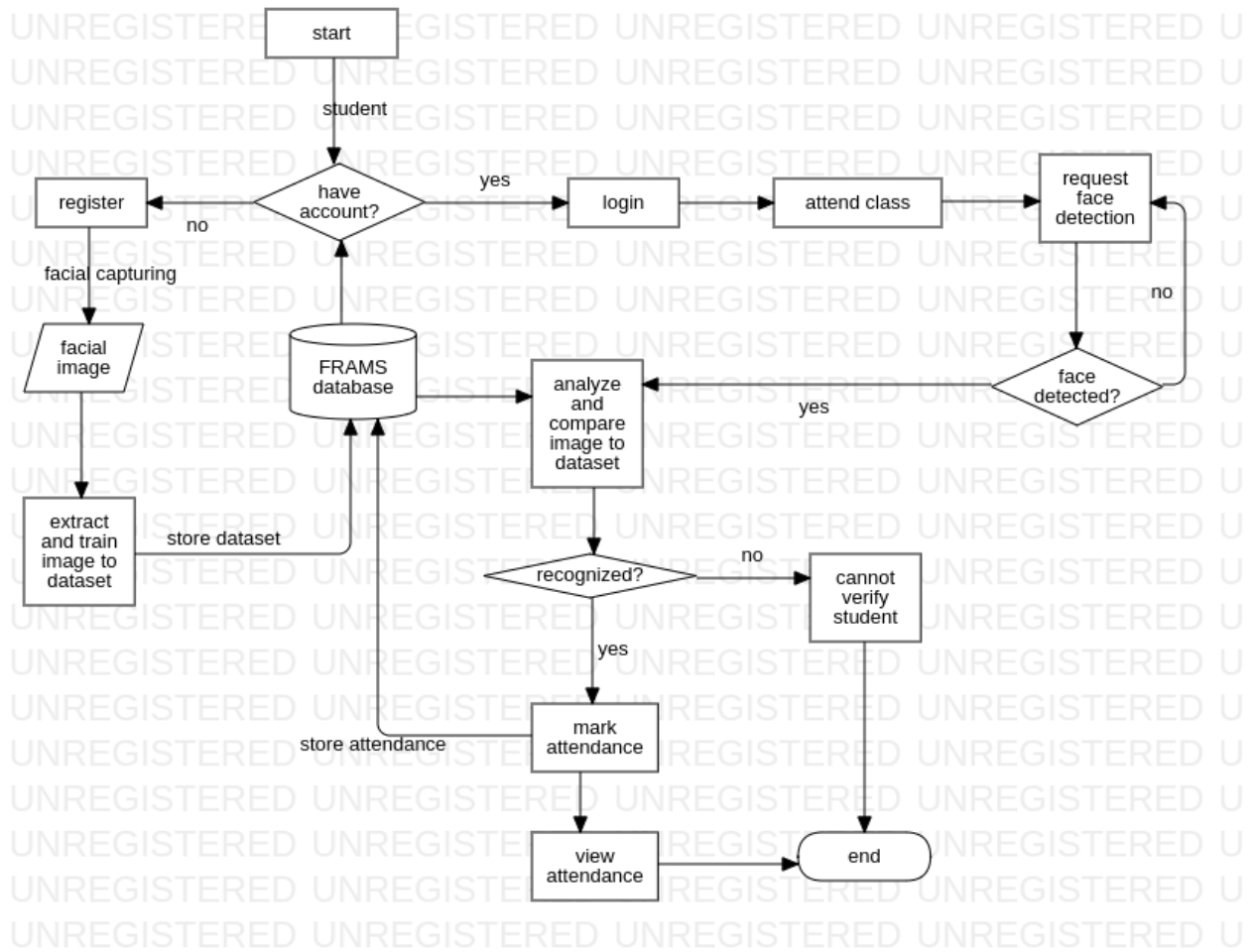


Figure 5-3 Student Flowchart

5.2.2.3. Staff flowchart

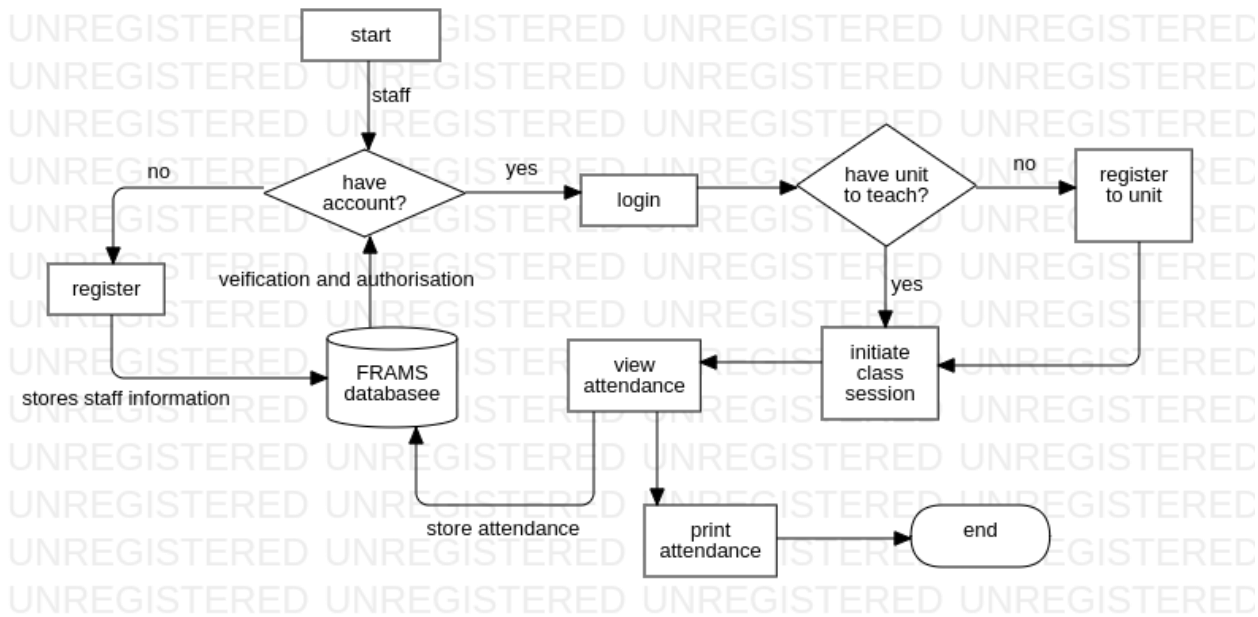


Figure 5-4 Staff Flowchart

5.2.3. Sequence Diagrams

The student sends a request for attendance to the system. The attendance server receives the request. The student's face image is captured by the facial recognition system. The server processes the image to extract facial features and identify the student.

The facial recognition model analyzes the image and compares it to the database of student facial features to identify the student. The response message is returned to the system indicating that the attendance has been recorded and the database has been updated. The attendance record is updated to reflect the student's attendance status. The attendance record is displayed to the user (e.g. student, teacher and admin) for verification or further action.

5.2.3.1. Staff Sequence Diagram

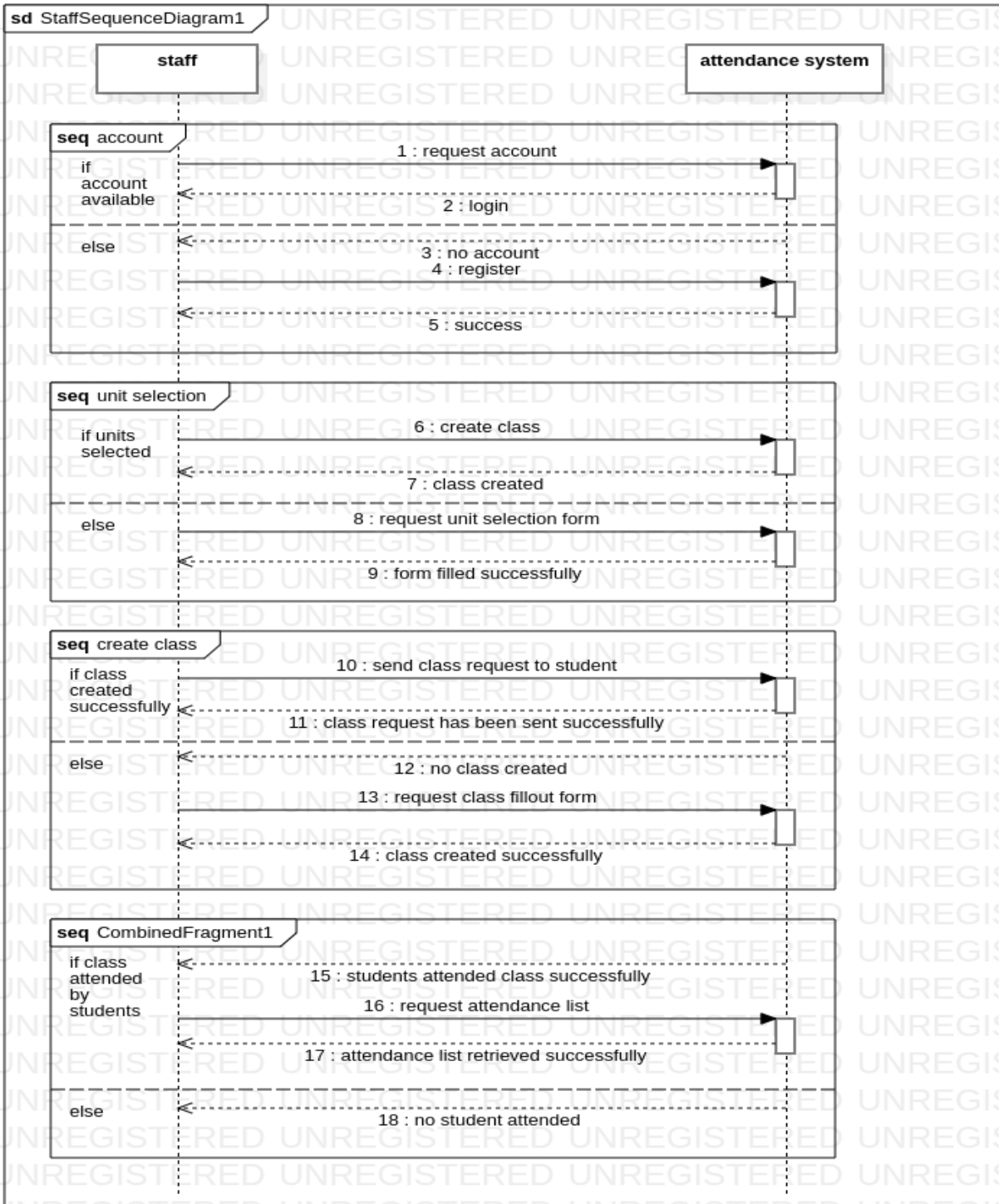


Figure 5-5 Staff Sequence diagram

5.2.3.2. Student Sequence Diagram

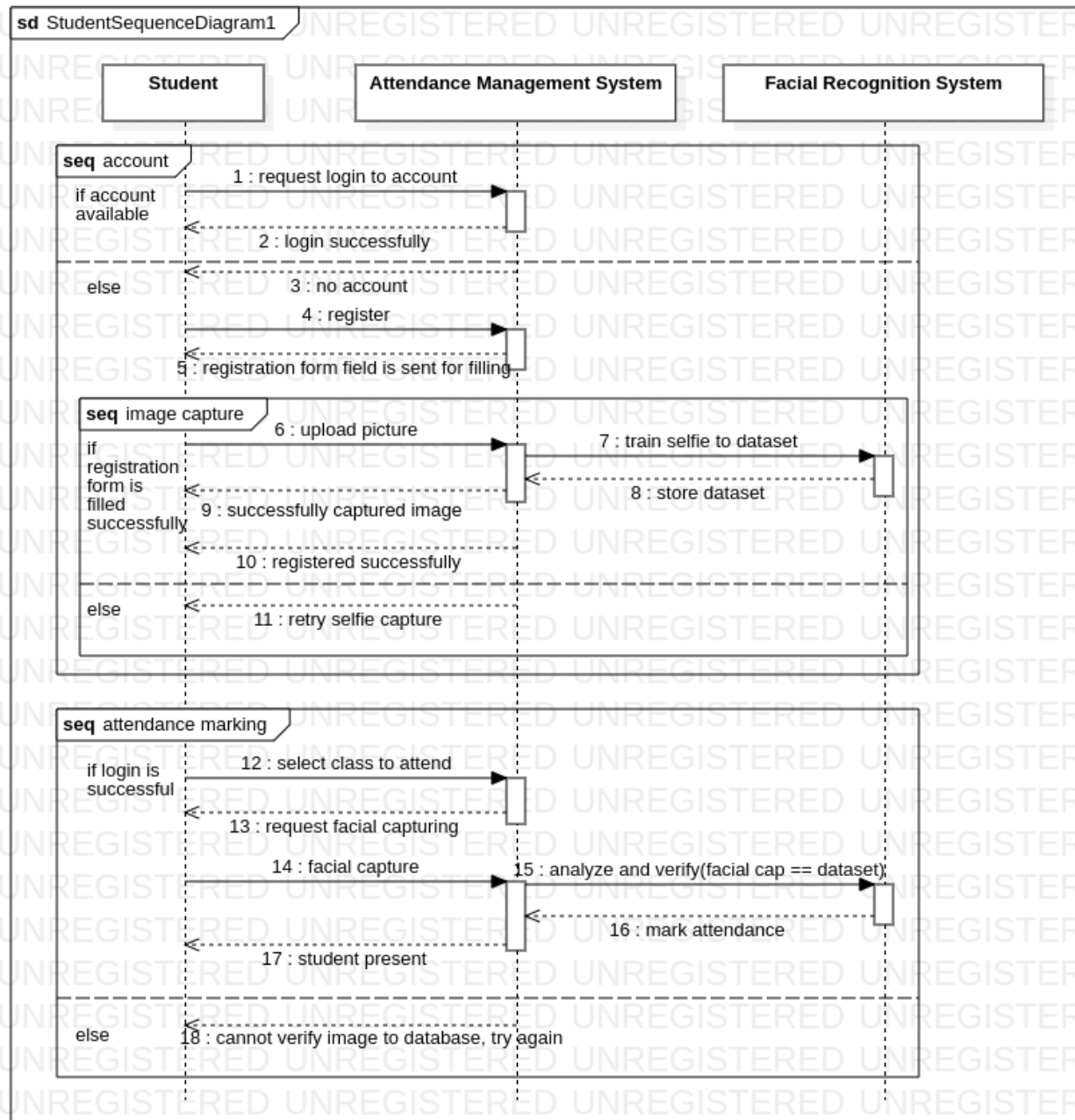


Figure 5-6 Student sequence diagram

5.2.3.3. Admin Sequence Diagram

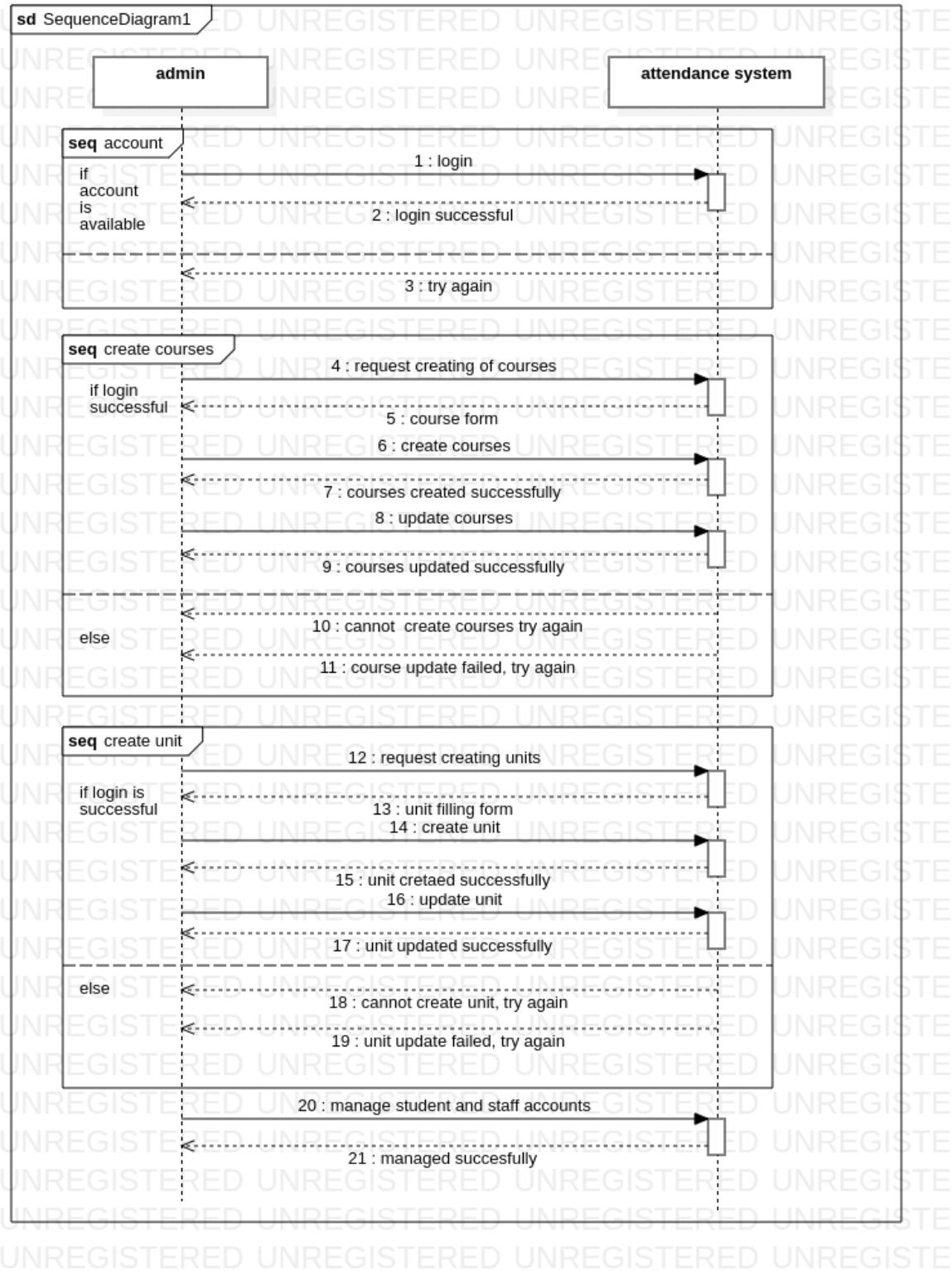


Figure 5-7 Admin sequence diagram

5.2.4. Context Diagram

5.2.4.1. (DFD level 0)

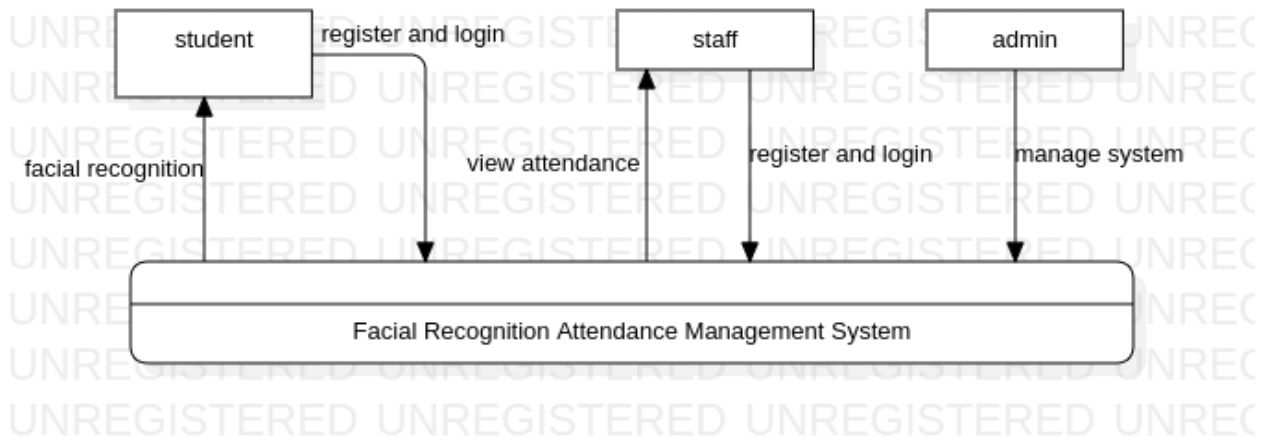


Figure 5-8 Data flow diagram Level 0

5.2.4.2. Data Flow Diagram (level 1)

This diagram shows the flow of data between the different modules and components of the facial recognition attendance management system for students.

In this system, the facial recognition system captures the facial images of students and sends them to the camera module. The camera module receives the images and forwards them to the face detection module, which detects and recognizes the faces of the students.

The student database module contains information about all registered students, including their personal information and biometric data. The attendance records module maintains a record of student attendance, which is updated in real-time as students are detected and recognized by the facial recognition system.

The notification module can be used to alert students or their parents if they are absent or late to class. The administrator panel allows the system administrator to manage and monitor the attendance system, including adding or removing students, generating reports, and configuring system settings.

This diagram shows the flow of data between the different modules and components of the facial recognition attendance management system for students.

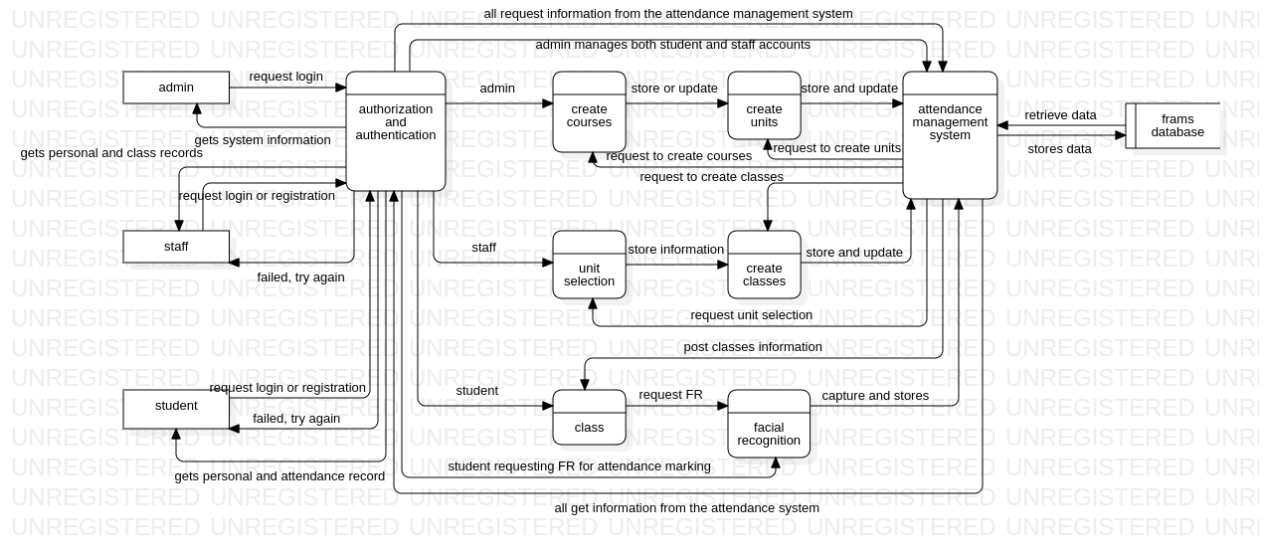


Figure 5-9 Data Flow diagram Level 1

5.2.5. Entity Relation Diagram (ERD)

In this ERD, there are three main entities: Student, Attendance, and Course.

The Student entity contains information about each student, including their unique `student_id`, `first_name`, `last_name`, `email`, and `photo_id`.

The Attendance entity contains information about each attendance record, including a unique `attendance_id`, the `student_id` of the student who attended, the `course_id` of the course they attended, the date of the attendance, and whether the student was present or not.

The Course entity contains information about each course, including a unique `course_id`, the `course_name`, the `start_time` and `end_time` of the course, and the `instructor_name` of the course.

The relationship between these entities is as follows:

A student can attend multiple courses, so there is a many-to-many relationship between the Student and Course entities. This is represented by a junction table with `student_id` and `course_id` as foreign keys.

Each student can have multiple attendance records, and each attendance record is associated with a single student and a single course. This is represented by foreign keys `student_id` and `course_id` in the Attendance entity.

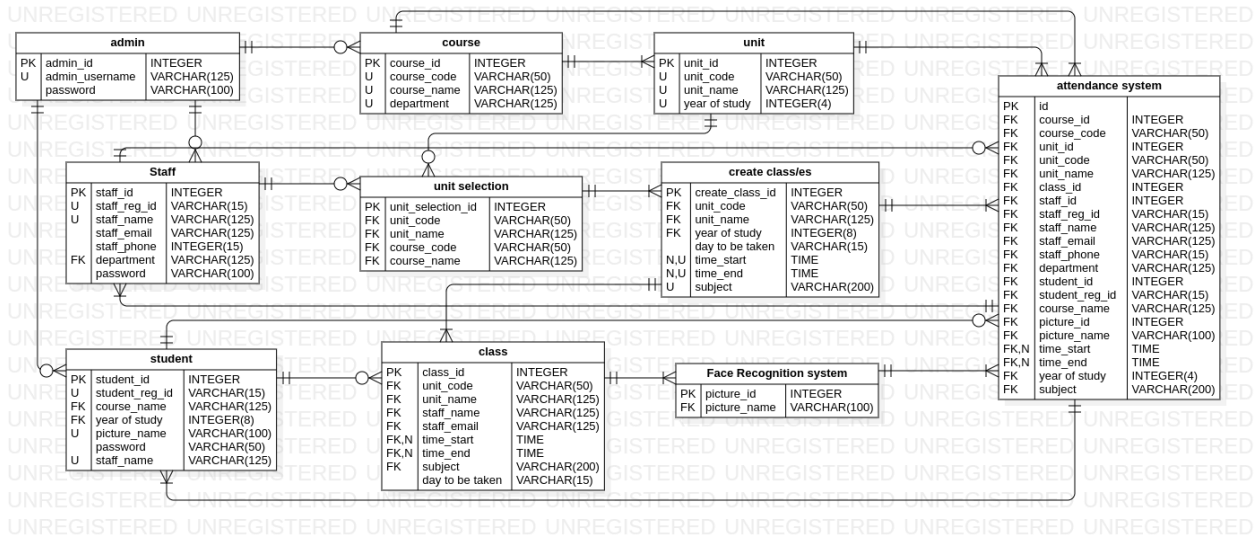


Figure 5-10 Entity Relation Diagram

6. IMPLIMENTATION.

Main System

App.py code that runs the whole system and

Our project Flask web application that creates and configures a SQLite database using SQLAlchemy, a popular Object Relational Mapper (ORM) for Python. The application defines several database models for different tables, including Students, Courses, Units, Staffs, Unitselection, and Createclasses, which represent different entities in the system. Each model is a subclass of **db.Model** and defines the fields (columns) for the respective database table.

The **Students** model represents student data and includes fields such as **id**, **firstName**, **lastName**, **regNumber**, **email**, **phone**, **school**, **profile_picture**, **passwd**, and **created_at**.

The **Courses** model represents course data and includes fields such as **id**, **school**, **ccode**, and **cname**.

The **Units** model represents unit data and includes fields such as **id**, **ccode**, **ucode**, **uname**, and **year**.

The **Staffs** model represents staff data and includes fields such as **id**, **firstName**, **lastName**, **eNumber**, **email**, **phone**, **sc**, **passwd**, and **created_at**.

The **Unitselection** model represents unit selection data and includes fields such as **id**, **ccode**, **ucode**, and **uname**.

The **Createclasses** model represents class creation data and includes fields such as **id**, **ccode**, **uname**, **year**, **day**, **time_start**, and **time_end**.

The code also configures the Flask application with the necessary settings for the database, such as the database URI, tracking modifications, and a secret key for session management. It initializes the SQLAlchemy extension with the Flask application using **db.init_app(app)**.

Finally, the code creates the database tables and inserts some initial data into the **Courses** and **Units** tables using SQLAlchemy's ORM. The **db.create_all()** method creates all the defined models as database tables, and the **db.session.add_all()** and **db.session.commit()** methods are used to add the initial data to the tables and commit the changes to the database.

PYTHON CODE FOR THE MAIN

```
##### Importing Libraries
#####
from flask import Flask, request, session, redirect, url_for, render_template, flash
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import func
from flask_login import UserMixin, login_manager, login_required, login_user, current_user,
logout_user
from werkzeug.security import generate_password_hash, check_password_hash
import os
from werkzeug.utils import secure_filename
import cv2
from datetime import datetime
import numpy as np
import face_recognition as face_rec
```

```
app = Flask(__name__)
```

```
basedir = os.path.abspath(os.path.dirname(__file__))
```

```
# Configuring database
db = SQLAlchemy()
app.config['SECRET_KEY'] = 'Nokeyisrequired'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database3.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db.init_app(app)
```

```
# Creating Staff Table in Database
```

```
class Staffs(db.Model, UserMixin):
    id = db.Column('staff_id', db.Integer, primary_key=True)
    firstName = db.Column(db.String(100), nullable=False)
    lastName = db.Column(db.String(100), nullable=False)
    eNumber = db.Column(db.String(100), unique=True, nullable=False)
    email = db.Column(db.String(80), unique=True, nullable=False)
    phone = db.Column(db.String(100), unique=True, nullable=False)
    school = db.Column(db.String(100), nullable=False)
    passwd = db.Column(db.String(100), nullable=False)
    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now())
```

```
def __repr__(self):
    return f'<Staff {self.firstName}>'
```

```
# Creating Student Table in Database
```

```

class Students(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    firstName = db.Column(db.String(100), nullable=False)
    lastName = db.Column(db.String(100), nullable=False)
    regNumber = db.Column(db.String(100), unique=True, nullable=False)
    email = db.Column(db.String(80), unique=True, nullable=False)
    phone = db.Column(db.String(100), unique=True, nullable=False)
    school = db.Column(db.String(100), nullable=False)
    course_id = db.Column(db.Integer, db.ForeignKey('courses.id'), nullable=False)
    profile_picture = db.Column(db.String(120), nullable=False)
    passwd = db.Column(db.String(80), nullable=False)
    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now())

    def __repr__(self):
        return f'<Student {self.firstName}>'

# Creating Courses Table in Database
class Courses(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    school = db.Column(db.String(100), nullable=False)
    ccode = db.Column(db.String(100), unique=True, nullable=False)
    cname = db.Column(db.String(100), nullable=False)
    units = db.relationship('Units', backref='course', lazy=True)

    def __repr__(self):
        return f'<Course {self.cname}>'

# Creating Units Table in Database
class Units(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey('courses.ccode'), nullable=False)
    ucode = db.Column(db.String(100), unique=True, nullable=False)
    uname = db.Column(db.String(100), nullable=False)
    year = db.Column(db.String(100), nullable=False)

    def __repr__(self):
        return f'<Unit {self.unit}>'

##### Creating Staff Table for unite selection in Database
#####
class Unitselection(db.Model):
    id = db.Column('unitselection_id', db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey(Units.ccode), nullable=False)
    ucode = db.Column(db.String(100), db.ForeignKey(Units.ucode), nullable=False)
    uname = db.Column(db.String(100), nullable=False)

```

```

def __repr__(self):
    return f'<Unitselection {self.unitselection_id}>'

##### Creating Staff Table for creating class in Database
#####
class Createclasses(db.Model):
    id = db.Column('createclass_id', db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey(Unitselection.ccode), nullable=False)
    uname = db.Column(db.String(100), db.ForeignKey(Unitselection.uname), nullable=False)
    year = db.Column(db.Integer, db.ForeignKey(Units.year), nullable=False)
    day = db.Column(db.VARCHAR(100), nullable=False)
    time_start = db.Column(db.Text)
    time_end = db.Column(db.Text)
    subject = db.Column(db.Text)

    def __repr__(self):
        return f'<Createclasses {self.createclass_id}>'

''' # Inserting data into Courses and Units tables
with app.app_context():
    db.create_all()

#Inserting data into uniselection table
    uniselection1 = Unitselection(ccode='I231',ucode='ITB_5205', uname='MULTIMEDIA
TECHNOLOGY')
    uniselection2 = Unitselection(ccode='I321', ucode='IBT_0304', uname='SYSTEM
DEVELOPMENT')
    uniselection3 = Unitselection(ccode='I132', ucode='ITB_2506', uname='MOBILE
COMMUNICATION')
    uniselection4 = Unitselection(ccode='ED', ucode='EDT_4307', uname='ENGILISHI')
    uniselection5 = Unitselection(ccode='ED', ucode='ECB_6306', uname='HUMAN
BEHAVIOR')
    db.session.add_all([uniselection1, uniselection2, uniselection3, uniselection4, uniselection5])
    db.session.commit()

#Inserting data into uniselection table
    class1 = Createclasses(ccode='I231',uname='SYSTEM
DEVELOPMENT',year='2.2',day='monday',time_start='10:00', time_end='12:00')
    class2 = Createclasses(ccode='I321',uname='IT
SECURITY',year='4.2',day='friday',time_start='10:00', time_end='12:00')
    class3 = Createclasses(ccode='EDS',uname='HUMAN
BEHAVIOR',year='3.2',day='tuesday',time_start='12:00', time_end='14:00')
    class4 =
Createclasses(ccode='ED',uname='ENGILISHI',year='1.2',day='monday',time_start='13:00',
time_end='15:00')

```



```
db.session.add_all([class1, class2, class3, class4])
db.session.commit()
```

```
# Inserting data into Courses table
```

```
course1 = Courses(school='SIIS', ccode='I231', cname='ICT')
course2 = Courses(school='SIIS', ccode='I321', cname='BIS')
course3 = Courses(school='SIIS', ccode='I132', cname='FCS')
course4 = Courses(school='EDUCATION', ccode='EDS', cname='EDUCATION SCIENCE')
course5 = Courses(school='EDUCATION', ccode='ED', cname='EDUCATION ARTS')
db.session.add_all([course1, course2, course3, course4, course5])
db.session.commit()
```

```
# Inserting data into Units table
```

```
unit1 = Units(ccode='I231', ucode='ITB_5205', uname='MULTIMEDIA TECHNOLOGY',
year='4.1')
```

```
unit2 = Units(ccode='I231', ucode='ITB_1203', uname='PROGRAMMING IN NET',
year='4.1')
```

```
unit3 = Units(ccode='I231', ucode='ITB_7204', uname='ADVANCED NETWORKIN',
year='4.1')
```

```
unit4 = Units(ccode='I321', ucode='IBT_0304', uname='SYSTEM DEVELOPMENT',
year='4.1')
```

```
unit5 = Units(ccode='I321', ucode='IBT_9305', uname='SYSTEM DESIGN', year='4.1')
```

```
unit6 = Units(ccode='I321', ucode='IBT_1306', uname='COMPUTER SYSTEM', year='4.1')
```

```
unit7 = Units(ccode='I132', ucode='ITB_2506', uname='MOBILE COMMUNICATION',
year='4.1')
```

```
unit8 = Units(ccode='I132', ucode='ITB_1507', uname='OPERATING SYSTEM', year='4.1')
```

```
unit9 = Units(ccode='I132', ucode='ITB_8508', uname='IT SECURITY', year='4.1')
```

```
unit10 = Units(ccode='EDS', ucode='EIT_3306', uname='KISWAHILI', year='4.1')
```

```
unit11 = Units(ccode='ED', ucode='EDT_4307', uname='ENGILISHI', year='4.1')
```

```
unit12 = Units(ccode='EDS', ucode='EIT_5308', uname='MULTIMEDIA', year='4.1')
```

```
unit13 = Units(ccode='ED', ucode='ECB_6306', uname='HUMAN BEHAVIOR', year='4.1')
```

```
db.session.add_all([unit1, unit2, unit3, unit4, unit5, unit6, unit7, unit8, unit9, unit10, unit11,
unit12, unit13])
```

```
db.session.commit()
```

```
#hassing the password
```

```
password = generate_password_hash('password') # Hash the passwords
```

```
# Inserting data into Students table
```

```

student1 = Students(firstName='John', lastName='Doe', regNumber='12345',
email='john.doe@example.com', phone='1234567890', school='SIIS', course_id=1,
profile_picture='profile.jpg', passwd=password) # Use hashed passwords
student2 = Students(firstName='Jane', lastName='Doe', regNumber='67890',
email='jane.doe@example.com', phone='9876543210', school='SIIS', course_id=2,
profile_picture='profile.jpg', passwd=password)
student3 = Students(firstName='kim', lastName='Doe', regNumber='57697',
email='kim.doe@example.com', phone='7654321098', school='EDUCATION', course_id=3,
profile_picture='profile.jpg', passwd=password)
student4 = Students(firstName='larry', lastName='Doe', regNumber='87993',
email='larry.doe@example.com', phone='0198765432', school='EDUCATION', course_id=4,
profile_picture='profile.jpg', passwd=password)
db.session.add_all([student1,student2,student3, student4])
db.session.commit()

#Inserting data into Staffs table
staff1 = Staffs(firstName='James', lastName='Doe', eNumber='12345',
email='john.doe@example.com', phone='1234567890', school='SIIS', passwd=password) # Use
hashed passwords
staff2 = Staffs(firstName='Jack', lastName='Doe', eNumber='67890',
email='jane.doe@example.com', phone='9876543210', school='SIIS', passwd=password)
staff3 = Staffs(firstName='Johe', lastName='Doe', eNumber='57697',
email='kim.doe@example.com', phone='7654321098', school='EDUCATION',
passwd=password)
staff4 = Staffs(firstName='Dison', lastName='Doe', eNumber='87993',
email='larry.doe@example.com', phone='0198765432', school='EDUCATION',
passwd=password)
db.session.add_all([staff1,staff2,staff3, staff4])
db.session.commit()'''

#Landing page for the root
@app.route('/', methods=['GET','POST'])
def index():
    return render_template('index.html')
##### Admin module Starts
#####
## Admin Login Starts
@app.route('/adminlogin/', methods=['GET','POST']) # Admin root
def adminlogin():
    if request.method == 'POST':
        username = request.form['username']
        passwd = request.form['passwd']
        # Perform login validation,
        if username == 'Admin' and passwd == 'admin123':
            # If login is successful, store username in session
            session['username'] = username

```

```

        flash('Login successful!', category='success')
        return redirect('/adminhome/') # Redirect to index page after successful login
    else:
        return render_template('admin/login.html', error='username or password is incorrect')
    return render_template('admin/login.html')
## Admin Home Starts
@app.route('/adminhome/')
def adminhome():
    if 'username' not in session:
        return render_template('admin/login.html')
    return render_template('admin/index.html')
## Admin create new student
@app.route('/createstudent/', methods=('GET', 'POST'))
def createstudent():
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        if request.method == 'POST':
            firstName = request.form['firstName']
            lastName = request.form['lastName']
            regNumber = request.form['regNumber']
            email = request.form['email']
            phone = request.form['phone']
            school = request.form['school']
            course_id = request.form['course_id']
            passwd1 = request.form['passwd']
            profile_picture = request.files['profile_picture']

            # Save profile picture to server
            filename = secure_filename(regNumber + '_' + profile_picture.filename) # Append
            regNumber to original filename
            profile_picture.save('static/uploads/' + filename)

            student = Students(firstName=firstName,
                               lastName=lastName,
                               regNumber=regNumber,
                               email=email,
                               phone=phone,
                               school=school,
                               course_id=course_id,
                               profile_picture=filename,
                               passwd=generate_password_hash(passwd1, method='sha256'))
            db.session.add(student)
            db.session.commit()
            return redirect(url_for('allstudents'))
    return render_template('admin/createstudent.html')

```

```

## Admin creat new staff
@app.route('/createstaff/', methods=('GET', 'POST'))
def createstaff():
    if 'username' not in session:
        return render_template('admin/login.html')
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        eNumber = request.form['eNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        passwd1 = request.form['passwd']

        staff = Staffs(firstName=firstName,
                        lastName=lastName,
                        eNumber=eNumber,
                        email=email,
                        phone=phone,
                        school=school,
                        passwd=generate_password_hash(passwd1, method='sha256'))
        db.session.add(staff)
        db.session.commit()
        return redirect(url_for('allstaffs'))
    else:
        return render_template('admin/createstaff.html')
## Admin view all students
@app.route('/allstudents/')
def allstudents():
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        students = Students.query.all()
        return render_template('admin/students.html', students=students)
## Admin view all staff
@app.route('/allstaffs/')
def allstaffs():
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staffs = Staffs.query.all()
        return render_template('admin/staffs.html', staffs=staffs)
## Admin per student view
@app.route('/studentid/<int:student_id>/')
def student(student_id):
    if 'username' not in session:

```

```

        return render_template('admin/login.html')
    else:
        student = Students.query.get_or_404(student_id)
        return render_template('admin/student.html', student=student)
## Admin per staff view
@app.route('/staffid/<int:staff_id>/')
def staff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        return render_template('admin/staff.html', staff=staff)
## Admin edit student function
@app.route('/studentid/<int:student_id>/edit/', methods=('GET', 'POST'))
def editstudent(student_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        student = Students.query.get_or_404(student_id)
        if request.method == 'POST':
            firstName = request.form['firstName']
            lastName = request.form['lastName']
            regNumber = request.form['regNumber']
            email = request.form['email']
            phone = request.form['phone']
            school = request.form['school']
            passwd1 = request.form['passwd']

            student.firstName = firstName
            student.lastName = lastName
            student.regNumber = regNumber
            student.email = email
            student.phone = phone
            student.school = school
            student.passwd = passwd1

            db.session.add(student)
            db.session.commit()
            return redirect(url_for('allstudents'))
        else:
            return render_template('admin/edit.html', student=student)
## Admin edit staff function
@app.route('/staffid/<int:staff_id>/editstaff/', methods=('GET', 'POST'))
def editstaff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')

```

```

else:
    staff = Staffs.query.get_or_404(staff_id)
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        eNumber = request.form['eNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        passwd1 = request.form['passwd']

        staff.firstName = firstName
        staff.lastName = lastName
        staff.eNumber = eNumber
        staff.email = email
        staff.phone = phone
        staff.school = school
        staff.passwd = passwd1

        db.session.add(staff)
        db.session.commit()
        return redirect(url_for('staffs'))
    else:
        return render_template('admin/editstaff.html', staff=staff)
## Admin delete student function
@app.post('/studentid/<int:student_id>/delete/')
def deletestudent(student_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        student = Students.query.get_or_404(student_id)
        db.session.delete(student)
        db.session.commit()
        return redirect(url_for('students'))
## Admin delete Staff function
@app.post('/staffid/<int:staff_id>/delete/')
def deletestaff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        db.session.delete(staff)
        db.session.commit()
        return redirect(url_for('staffs'))

```

```
##### Admin courses directory Starts
#####
@app.route('/courses/')
def courses():
    if 'username' not in session:
        return render_template('admin/login.html')
    courses = Courses.query.all()
    return render_template('admin/courses.html', courses=courses)

##### Admin units directory Starts
#####
@app.route('/units/')
def units():
    if 'username' not in session:
        return render_template('admin/login.html')
    units = Units.query.all()
    return render_template('admin/units.html', units=units)

##### Admin add course directory Starts
#####
@app.route('/createcourse/', methods=('GET', 'POST'))
def createcourse():
    if 'username' not in session:
        return render_template('admin/login.html')
    if request.method == 'POST':
        ccode = request.form['ccode']
        cname = request.form['cname']
        course = Courses(ccode=ccode, cname=cname)

        db.session.add(course)
        db.session.commit()
        return redirect(url_for('courses'))
    return render_template('admin/createcourse.html')

##### Admin add unit directory Starts
#####
@app.route('/createunit/', methods=('GET', 'POST'))
def createunit():
    if 'username' not in session:
        return render_template('admin/login.html')
    courses = Courses.query.all()
    if request.method == 'POST':
        ccode = request.form['ccode']
        ucode = request.form['ucode']
        uname = request.form['uname']
        year = request.form['year']
```

```

unit = Units(ccode=ccode, ucode=ucode, uname=uname, year=year)

db.session.add(unit)
db.session.commit()
return redirect(url_for('units'))
return render_template('admin/createunit.html', courses =courses)

##### Admin module Ends
#####

##### Staff module Starts
#####

#### Staff register
@app.route('/sregister/', methods=['GET', 'POST'])
def staffregister():
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        eNumber = request.form['eNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        passwd = request.form['passwd']

        new_staff = Staffs(firstName=firstName,
                           lastName=lastName,
                           eNumber=eNumber,
                           email=email,
                           phone=phone,
                           school=school,
                           passwd=generate_password_hash(passwd, method='sha256'))
        db.session.add(new_staff)
        db.session.commit()

        return redirect(url_for('stafflogin'))
    return render_template('staff/register.html')
#### Staff login
@app.route('/slogin/', methods=['GET', 'POST'])
def stafflogin():
    if request.method == 'POST':
        # Get student input from form
        eNumber = request.form['eNumber']
        passwd = request.form['passwd']
        # Perform login validation, e.g. check if regNumber and password match
        staff = Staffs.query.filter_by(eNumber=eNumber).first()
        if staff:

```



```

        if check_password_hash(staff.passwd, passwd):
            # If login is successful, store regNumber in session
            session['eNumber'] = eNumber
            flash('Login successful!', category='success')
            return redirect(url_for('staffhome')) # Redirect to index page after successful login
    return render_template('staff/login.html')
#### Staff Homepage
@app.route('/staffHome/')
def staffhome():
    # Check if student is logged in
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    return render_template('staff/home.html')
#### Staff staff attendance side
@app.route('/studentattendance/')
def studentattendance():
    # Check if student is logged in
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    return render_template('staff/attendance.html')
#### Staff create class side
@app.route('/createclass/', methods=['GET', 'POST'])
def createclass():
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    units = Units.query.all()
    courses = Courses.query.all()

    if request.method == 'POST':
        ccode = request.form['ccode']
        uname = request.form['uname']
        year = request.form['year']
        day = request.form['day']
        time_start = request.form['time_start']
        time_end = request.form['time_end']
        subject = request.form['subject']

        new_createclass = Createclasses(ccode=ccode, uname=uname, year=year, day=day,
time_start=time_start, time_end=time_end, subject=subject)
        db.session.add(new_createclass)
        db.session.commit()
        return redirect(url_for('classes'))
    return render_template('staff/createclass.html', courses=courses, units=units)
#### Staff class side
@app.route('/classes/', methods=['GET', 'POST'])
def classes():

```

```

cclasses = Createclasses.query.all()
if 'eNumber' not in session:
    return redirect('/slogin/') # Redirect to login page if student is not logged in

    return render_template('staff/classes.html', cclasses=cclasses)
#### Staff selectioning side
@app.route('/unitselection/', methods=['GET', 'POST'])
def unitselection():
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    else:
        units = Units.query.all()
        courses = Courses.query.all()
    return render_template('staff/unitselection.html' ,units=units,courses=courses)

@app.route('/search', methods=['post'])
def search():
    if 'eNumber' not in session:
        return redirect('/slogin/')
    keyword = request.form['keyword']
    results = Courses.query.filter(Courses.all.contains(keyword)).all()

    return render_template('staff/unitselection.html', results=results)
##### Staff module Ends
#####

##### student module starts
#####
# Route for registration form
@app.route('/studentregister/', methods=('GET', 'POST'))
def studentregister():
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        regNumber = request.form['regNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        course_id =request.form['course_id']
        passwd1 =request.form['passwd']
        profile_picture = request.files['profile_picture']

        # Save profile picture to server
        filename = secure_filename(regNumber + '_' + profile_picture.filename) # Append
regNumber to original filename
        profile_picture.save('static/uploads/' + filename)

```

```

        student = Students(firstName=firstName,
                            lastName=lastName,
                            regNumber=regNumber,
                            email=email,
                            phone=phone,
                            school=school,
                            course_id=course_id,
                            profile_picture=filename,
                            passwd=generate_password_hash(passwd1, method='sha256'))
        db.session.add(student)
        db.session.commit()
        return redirect('/studentlogin/') # Redirect to login page after successful registration
    return render_template('register.html')
# Route for login form
@app.route('/studentlogin/', methods=('GET', 'POST'))
def studentlogin():
    if request.method == 'POST':
        # Get student input from form
        regNumber = request.form['regNumber']
        passwd = request.form['passwd']
        # Perform login validation, e.g. check if regNumber and password match
        student = Students.query.filter_by(regNumber=regNumber).first()
        if student:
            if check_password_hash(student.passwd, passwd):
                # If login is successful, store regNumber in session
                session['regNumber'] = regNumber
                flash('Login successful!', category='success')
                return redirect('/studentpage/') # Redirect to index page after successful login
            else:
                return render_template('login.html', error='Invalid Registration Number or password')
        return render_template('login.html')
# student index page
@app.route('/studentpage/')
def studentpage():
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in
    else:
        students = Students.query.all()
        return render_template('home.html', students=students)
# student attendance page
@app.route('/studentsattendance/')
def studentsattendance():
    # Check if student is logged in
    if 'regNumber' not in session:

```

```

        return redirect('/studentlogin/') # Redirect to login page if student is not logged in
    else:
        return render_template('studentattendance.html')
# student classes page
@app.route('/studentclasses/')
def studentclasses():
    cclasses = Createclasses.query.all()
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in

    return render_template('studentclasses.html', cclasses=cclasses)

@app.route('/studentmarkattendance/')
def takeattendance():
    def resize(img, size):
        width = int(img.shape[1]*size)
        height = int(img.shape[0]*size)
        dimension = (width, height)
        return cv2.resize(img, dimension, interpolation=cv2.INTER_AREA)

    path = "static/uploads"
    studentImg = []
    studentName = []
    myList = os.listdir(path)
    print(myList)

    for cl in myList :
        curImg = cv2.imread(f'{path}/{cl}') # loops in the path student_images/. from the first
        image to the last
        studentImg.append(curImg) #it reads all images saved in the student_images and adds it to
        the studentImg list
        studentName.append(os.path.splitext(cl)[0]) #it will splite the name the image is saved with
        from the extension".jpg" and saves the name to studentName without extension
        print(studentName)

    def findEncoding(images) : #function to encode the emages in the student_images dir
        imgEncodings = [] # create a list to store all our encodings
        for img in images : # Loop for getting encodings from each image
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            encodeimg = face_rec.face_encodings(img)[0] #add 0 if its an image not a video
            imgEncodings.append(encodeimg)
        return imgEncodings

```

EncodeList = findEncoding(studentImg) #find the encodings of student image which are all the images in studentsimg that we retrived from the values in the for Loop

```
def MarkAttendance(name):
    with open('attendance/attendance.csv', 'r+') as f: # read and write the file the name of
student and the time they entered th class
        myDataList = f.readlines()
        nameList = []
        for line in myDataList :
            entry = line.split(',')
            nameList.append(entry[0])

        if name not in nameList :
            now = datetime.now()
            timestr = now.strftime('%H : %M')
            f.writelines(f'\n{name}, {timestr}')
            statment = str('Welcome to class' + name)

#getting video from webcam
vid = cv2.VideoCapture(0)

while True :
    # get each frame individualy
    success, frame = vid.read()
    # resize the frames
    smaller_frames = cv2.resize(frame, (0,0), None, 0.25, 0.25)
    # convert color
    smaller_frames = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    # detect all the faces in one frame
    facesInFrame = face_rec.face_locations(smaller_frames)
    # encode all faces in the frame
    encodeFacesInFrame = face_rec.face_encodings(smaller_frames, facesInFrame)

    #loop that allows to compare all the encodings of faces in the frams with encodings of our
saved pictures
    for encodeFace, faceloc in zip(encodeFacesInFrame, facesInFrame) : # we are using zip
because we are comparing two files in one loop
        matches = face_rec.compare_faces(EncodeList, encodeFace)
        facedis = face_rec.face_distance(EncodeList, encodeFace)
        #print(facedis)
        matchIndex = np.argmin(facedis)

        if matches[matchIndex] :
            name = studentName[matchIndex].upper()
            y1, x2, y2, x1 = faceloc
```

```

        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 3)
        cv2.rectangle(frame, (x1, y2-25), (x2, y2), (0, 255, 0), cv2.FILLED)
        cv2.putText(frame, name, (x1+6, y2-6), cv2.FONT_HERSHEY_COMPLEX, 1, (255,
255, 255), 2)
        MarkAttendance(name)

        #print(name)

    cv2.imshow('video', frame)
    if cv2.waitKey(1) & 0xFF ==ord('q') :
        break

    cap.release()
    cv2.destroyAllWindows()

return render_template('studentclasses.html')

```

```

# Log out route
@app.route('/logout/', methods=['POST'])
def logout():
    # Clear the session
    session.clear()
    flash('You have been logged out.', 'success')
    # Redirect to login page after logging out
    return redirect('/')
##### Student module ends
#####

if __name__ == '__main__':
    app.run(debug=True)##### Importing Libraries
#####
from flask import Flask, request, session, redirect, url_for, render_template, flash
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import func
from flask_login import UserMixin, login_manager, login_required, login_user, current_user,
logout_user
from werkzeug.security import generate_password_hash, check_password_hash
import os
from werkzeug.utils import secure_filename
import cv2
from datetime import datetime

```

```

import numpy as np
import face_recognition as face_rec

app = Flask(__name__)

basedir = os.path.abspath(os.path.dirname(__file__))

# Configuring database
db = SQLAlchemy()
app.config['SECRET_KEY'] = 'Nokeyisrequired'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database3.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db.init_app(app)

# Creating Staff Table in Database
class Staffs(db.Model, UserMixin):
    id = db.Column('staff_id', db.Integer, primary_key=True)
    firstName = db.Column(db.String(100), nullable=False)
    lastName = db.Column(db.String(100), nullable=False)
    eNumber = db.Column(db.String(100), unique=True, nullable=False)
    email = db.Column(db.String(80), unique=True, nullable=False)
    phone = db.Column(db.String(100), unique=True, nullable=False)
    school = db.Column(db.String(100), nullable=False)
    passwd = db.Column(db.String(100), nullable=False)
    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now())

    def __repr__(self):
        return f'<Staff {self.firstName}>'

# Creating Student Table in Database
class Students(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    firstName = db.Column(db.String(100), nullable=False)
    lastName = db.Column(db.String(100), nullable=False)
    regNumber = db.Column(db.String(100), unique=True, nullable=False)
    email = db.Column(db.String(80), unique=True, nullable=False)
    phone = db.Column(db.String(100), unique=True, nullable=False)
    school = db.Column(db.String(100), nullable=False)
    course_id = db.Column(db.Integer, db.ForeignKey('courses.id'), nullable=False)
    profile_picture = db.Column(db.String(120), nullable=False)
    passwd = db.Column(db.String(80), nullable=False)
    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now())

```

```

def __repr__(self):
    return f'<Student {self.firstName}>'

# Creating Courses Table in Database
class Courses(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    school = db.Column(db.String(100), nullable=False)
    ccode = db.Column(db.String(100), unique=True, nullable=False)
    cname = db.Column(db.String(100), nullable=False)
    units = db.relationship('Units', backref='course', lazy=True)

    def __repr__(self):
        return f'<Course {self.cname}>'

# Creating Units Table in Database
class Units(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey('courses.ccode'), nullable=False)
    ucode = db.Column(db.String(100), unique=True, nullable=False)
    uname = db.Column(db.String(100), nullable=False)
    year = db.Column(db.String(100), nullable=False)

    def __repr__(self):
        return f'<Unit {self.unit}>'

##### Creating Staff Table for unite selection in Database
#####
class Unitselection(db.Model):
    id = db.Column('unitselection_id', db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey(Units.ccode), nullable=False)
    ucode = db.Column(db.String(100), db.ForeignKey(Units.ucode), nullable=False)
    uname = db.Column(db.String(100), nullable=False)

    def __repr__(self):
        return f'<Unitselection {self.unitselection_id}>'

##### Creating Staff Table for creating class in Database
#####
class Createclasses(db.Model):
    id = db.Column('createclass_id', db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey(Unitselection.ccode), nullable=False)
    uname = db.Column(db.String(100), db.ForeignKey(Unitselection.uname), nullable=False)
    year = db.Column(db.Integer, db.ForeignKey(Units.year), nullable=False)
    day = db.Column(db.VARCHAR(100), nullable=False)
    time_start = db.Column(db.Text)
    time_end = db.Column(db.Text)

```



```

subject = db.Column(db.Text)

def __repr__(self):
    return f'<Createclasses {self.createclass_id}>'

''' # Inserting data into Courses and Units tables
with app.app_context():
    db.create_all()

#Inserting data into uniselection table
    uniselection1 = Unitselection(ccode='I231',ucode='ITB_5205', uname='MULTIMEDIA
TECHNOLOGY')
    uniselection2 = Unitselection(ccode='I321', ucode='IBT_0304', uname='SYSTEM
DEVELOPMENT')
    uniselection3 = Unitselection(ccode='I132', ucode='ITB_2506', uname='MOBILE
COMMUNICATION')
    uniselection4 = Unitselection(ccode='ED', ucode='EDT_4307', uname='ENGILISHI')
    uniselection5 = Unitselection(ccode='ED', ucode='ECB_6306', uname='HUMAN
BEHAVIOR')
    db.session.add_all([uniselection1, uniselection2, uniselection3, uniselection4, uniselection5])
    db.session.commit()

#Inserting data into uniselection table
    class1 = Createclasses(ccode='I231',uname='SYSTEM
DEVELOPMENT',year='2.2',day='monday',time_start='10:00', time_end='12:00')
    class2 = Createclasses(ccode='I321',uname='IT
SECURITY',year='4.2',day='friday',time_start='10:00', time_end='12:00')
    class3 = Createclasses(ccode='EDS',uname='HUMAN
BEHAVIOR',year='3.2',day='tuesday',time_start='12:00', time_end='14:00')
    class4 =
Createclasses(ccode='ED',uname='ENGILISHI',year='1.2',day='monday',time_start='13:00',
time_end='15:00')

    db.session.add_all([class1, class2, class3, class4])
    db.session.commit()

# Inserting data into Courses table
course1 = Courses(school='SIIS', ccode='I231', cname='ICT')
course2 = Courses(school='SIIS', ccode='I321', cname='BIS')
course3 = Courses(school='SIIS', ccode='I132', cname='FCS')
course4 = Courses(school='EDUCATION', ccode='EDS', cname='EDUCATION SCIENCE')
course5 = Courses(school='EDUCATION', ccode='ED', cname='EDUCATION ARTS')
db.session.add_all([course1, course2, course3, course4, course5])
db.session.commit()

```

```

# Inserting data into Units table
unit1 = Units(ccode='I231', ucode='ITB_5205', uname='MULTIMEDIA TECHNOLOGY',
year='4.1')
unit2 = Units(ccode='I231', ucode='ITB_1203', uname='PROGRAMMING IN NET',
year='4.1')
unit3 = Units(ccode='I231', ucode='ITB_7204', uname='ADVANCED NETWORKIN',
year='4.1')

unit4 = Units(ccode='I321', ucode='IBT_0304', uname='SYSTEM DEVELOPMENT',
year='4.1')
unit5 = Units(ccode='I321', ucode='IBT_9305', uname='SYSTEM DESIGN', year='4.1')
unit6 = Units(ccode='I321', ucode='IBT_1306', uname='COMPUTER SYSTEM', year='4.1')

unit7 = Units(ccode='I132', ucode='ITB_2506', uname='MOBILE COMMUNICATION',
year='4.1')
unit8 = Units(ccode='I132', ucode='ITB_1507', uname='OPERATING SYSTEM', year='4.1')
unit9 = Units(ccode='I132', ucode='ITB_8508', uname='IT SECURITY', year='4.1')

unit10 = Units(ccode='EDS', ucode='EIT_3306', uname='KISWAHILI', year='4.1')
unit11 = Units(ccode='ED', ucode='EDT_4307', uname='ENGILISHI', year='4.1')
unit12 = Units(ccode='EDS', ucode='EIT_5308', uname='MULTIMEDIA', year='4.1')
unit13 = Units(ccode='ED', ucode='ECB_6306', uname='HUMAN BEHAVIOR', year='4.1')

db.session.add_all([unit1, unit2, unit3, unit4, unit5, unit6, unit7, unit8, unit9, unit10, unit11,
unit12, unit13])
db.session.commit()

#hassing the password
password = generate_password_hash('password') # Hash the passwords
# Inserting data into Students table
student1 = Students(firstName='John', lastName='Doe', regNumber='12345',
email='john.doe@example.com', phone='1234567890', school='SIIS', course_id=1,
profile_picture='profile.jpg', passwd=password) # Use hashed passwords
student2 = Students(firstName='Jane', lastName='Doe', regNumber='67890',
email='jane.doe@example.com', phone='9876543210', school='SIIS', course_id=2,
profile_picture='profile.jpg', passwd=password)
student3 = Students(firstName='kim', lastName='Doe', regNumber='57697',
email='kim.doe@example.com', phone='7654321098', school='EDUCATION', course_id=3,
profile_picture='profile.jpg', passwd=password)
student4 = Students(firstName='larry', lastName='Doe', regNumber='87993',
email='larry.doe@example.com', phone='0198765432', school='EDUCATION', course_id=4,
profile_picture='profile.jpg', passwd=password)
db.session.add_all([student1, student2, student3, student4])
db.session.commit()

```

```

#Inserting data into Staffs table
staff1 = Staffs(firstName='James', lastName='Doe', eNumber='12345',
email='john.doe@example.com', phone='1234567890', school='SIIS', passwd=password) # Use
hashed passwords
staff2 = Staffs(firstName='Jack', lastName='Doe', eNumber='67890',
email='jane.doe@example.com', phone='9876543210', school='SIIS', passwd=password)
staff3 = Staffs(firstName='Johe', lastName='Doe', eNumber='57697',
email='kim.doe@example.com', phone='7654321098', school='EDUCATION',
passwd=password)
staff4 = Staffs(firstName='Dison', lastName='Doe', eNumber='87993',
email='larry.doe@example.com', phone='0198765432', school='EDUCATION',
passwd=password)
db.session.add_all([staff1,staff2,staff3, staff4])
db.session.commit()

#Landing page for the root
@app.route('/', methods=['GET','POST'])
def index():
    return render_template('index.html')
##### Admin module Starts
#####
## Admin Login Starts
@app.route('/adminlogin/', methods=['GET','POST']) # Admin root
def adminlogin():
    if request.method == 'POST':
        username = request.form['username']
        passwd = request.form['passwd']
        # Perform login validation,
        if username == 'Admin' and passwd == 'admin123':
            # If login is successful, store username in session
            session['username'] = username
            flash('Login successful!', category='success')
            return redirect('/adminhome/') # Redirect to index page after successful login
        else:
            return render_template('admin/login.html', error='username or password is incorrect')
    return render_template('admin/login.html')
## Admin Home Starts
@app.route('/adminhome/')
def adminhome():
    if 'username' not in session:
        return render_template('admin/login.html')
    return render_template('admin/index.html')
## Admin create new student
@app.route('/createstudent/', methods=('GET', 'POST'))
def createstudent():
    if 'username' not in session:

```

```

        return render_template('admin/login.html')
    else:
        if request.method == 'POST':
            firstName = request.form['firstName']
            lastName = request.form['lastName']
            regNumber = request.form['regNumber']
            email = request.form['email']
            phone = request.form['phone']
            school = request.form['school']
            course_id = request.form['course_id']
            passwd1 = request.form['passwd']
            profile_picture = request.files['profile_picture']

            # Save profile picture to server
            filename = secure_filename(regNumber + '_' + profile_picture.filename) # Append
            regNumber to original filename
            profile_picture.save('static/uploads/' + filename)

            student = Students(firstName=firstName,
                               lastName=lastName,
                               regNumber=regNumber,
                               email=email,
                               phone=phone,
                               school=school,
                               course_id=course_id,
                               profile_picture=filename,
                               passwd=generate_password_hash(passwd1, method='sha256'))
            db.session.add(student)
            db.session.commit()
            return redirect(url_for('allstudents'))
        return render_template('admin/createstudent.html')
## Admin creat new staff
@app.route('/createstaff/', methods=('GET', 'POST'))
def createstaff():
    if 'username' not in session:
        return render_template('admin/login.html')
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        eNumber = request.form['eNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        passwd1 = request.form['passwd']

        staff = Staffs(firstName=firstName,

```

```

        lastName=lastName,
        eNumber=eNumber,
        email=email,
        phone=phone,
        school=school,
        passwd=generate_password_hash(passwd1, method='sha256'))
    db.session.add(staff)
    db.session.commit()
    return redirect(url_for('allstaffs'))
else:
    return render_template('admin/createstaff.html')
## Admin view all students
@app.route('/allstudents/')
def allstudents():
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        students = Students.query.all()
        return render_template('admin/students.html', students=students)
## Admin view all staff
@app.route('/allstaffs/')
def allstaffs():
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staffs = Staffs.query.all()
        return render_template('admin/staffs.html', staffs=staffs)
## Admin per student view
@app.route('/studentid/<int:student_id>/')
def student(student_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        student = Students.query.get_or_404(student_id)
        return render_template('admin/student.html', student=student)
## Admin per staff view
@app.route('/staffid/<int:staff_id>/')
def staff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        return render_template('admin/staff.html', staff=staff)
## Admin edit student function
@app.route('/studentid/<int:student_id>/edit/', methods=('GET', 'POST'))
def editstudent(student_id):

```

```

if 'username' not in session:
    return render_template('admin/login.html')
else:
    student = Students.query.get_or_404(student_id)
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        regNumber = request.form['regNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        passwd1 = request.form['passwd']

        student.firstName = firstName
        student.lastName = lastName
        student.regNumber = regNumber
        student.email = email
        student.phone = phone
        student.school = school
        student.passwd = passwd1

        db.session.add(student)
        db.session.commit()
        return redirect(url_for('allstudents'))
    else:
        return render_template('admin/edit.html', student=student)
## Admin edit staff function
@app.route('/staffid/<int:staff_id>/editstaff/', methods=('GET', 'POST'))
def editstaff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        if request.method == 'POST':
            firstName = request.form['firstName']
            lastName = request.form['lastName']
            eNumber = request.form['eNumber']
            email = request.form['email']
            phone = request.form['phone']
            school = request.form['school']
            passwd1 = request.form['passwd']

            staff.firstName = firstName
            staff.lastName = lastName
            staff.eNumber = eNumber

```

```

        staff.email = email
        staff.phone = phone
        staff.school = school
        staff.passwd = passwd1

        db.session.add(staff)
        db.session.commit()
        return redirect(url_for('staffs'))
    else:
        return render_template('admin/editstaff.html',staff=staff)
## Admin delete student function
@app.post('/studentid/<int:student_id>/delete/')
def deletestudent(student_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        student = Students.query.get_or_404(student_id)
        db.session.delete(student)
        db.session.commit()
        return redirect(url_for('students'))
## Admin delete Staff function
@app.post('/staffid/<int:staff_id>/delete/')
def deletestaff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        db.session.delete(staff)
        db.session.commit()
        return redirect(url_for('staffs'))

##### Admin courses directory Starts
#####
@app.route('/courses/')
def courses():
    if 'username' not in session:
        return render_template('admin/login.html')
    courses = Courses.query.all()
    return render_template('admin/courses.html', courses=courses)

##### Admin units directory Starts
#####
@app.route('/units/')
def units():
    if 'username' not in session:
        return render_template('admin/login.html')

```

```

units = Units.query.all()
return render_template('admin/units.html', units=units)

##### Admin add course directory Starts
#####
@app.route('/createcourse/', methods=('GET', 'POST'))
def createcourse():
    if 'username' not in session:
        return render_template('admin/login.html')
    if request.method == 'POST':
        ccode = request.form['ccode']
        cname = request.form['cname']
        course = Courses(ccode=ccode, cname=cname)

        db.session.add(course)
        db.session.commit()
        return redirect(url_for('courses'))
    return render_template('admin/createcourse.html')

##### Admin add unit directory Starts
#####
@app.route('/createunit/', methods=('GET', 'POST'))
def createunit():
    if 'username' not in session:
        return render_template('admin/login.html')
    courses = Courses.query.all()
    if request.method == 'POST':
        ccode = request.form['ccode']
        ucode = request.form['ucode']
        uname = request.form['uname']
        year = request.form['year']
        unit = Units(ccode=ccode, ucode=ucode, uname=uname, year=year)

        db.session.add(unit)
        db.session.commit()
        return redirect(url_for('units'))
    return render_template('admin/createunit.html', courses =courses)

##### Admin module Ends
#####

##### Staff module Starts
#####
### Staff register
@app.route('/sregister/', methods=['GET', 'POST'])
def staffregister():

```



```

if request.method == 'POST':
    firstName = request.form['firstName']
    lastName = request.form['lastName']
    eNumber = request.form['eNumber']
    email = request.form['email']
    phone = request.form['phone']
    school = request.form['school']
    passwd = request.form['passwd']

    new_staff = Staffs(firstName=firstName,
                        lastName=lastName,
                        eNumber=eNumber,
                        email=email,
                        phone=phone,
                        school=school,
                        passwd=generate_password_hash(passwd, method='sha256'))
    db.session.add(new_staff)
    db.session.commit()

    return redirect(url_for('stafflogin'))
    return render_template('staff/register.html')
### Staff login
@app.route('/slogin/', methods=['GET', 'POST'])
def stafflogin():
    if request.method == 'POST':
        # Get student input from form
        eNumber = request.form['eNumber']
        passwd = request.form['passwd']
        # Perform login validation, e.g. check if regNumber and password match
        staff = Staffs.query.filter_by(eNumber=eNumber).first()
        if staff:
            if check_password_hash(staff.passwd, passwd):
                # If login is successful, store regNumber in session
                session['eNumber'] = eNumber
                flash('Login successful!', category='success')
                return redirect(url_for('staffhome')) # Redirect to index page after successful login
    return render_template('staff/login.html')
### Staff Homepage
@app.route('/staffHome/')
def staffhome():
    # Check if student is logged in
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    return render_template('staff/home.html')
### Staff staff attendance side
@app.route('/studentattendance/')

```

```

def studentattendance():
    # Check if student is logged in
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    return render_template('staff/attendance.html')
#### Staff create class side
@app.route('/createclass/', methods=['GET', 'POST'])
def createclass():
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    units = Units.query.all()
    courses = Courses.query.all()

    if request.method == 'POST':
        ccode = request.form['ccode']
        uname = request.form['uname']
        year = request.form['year']
        day = request.form['day']
        time_start = request.form['time_start']
        time_end = request.form['time_end']
        subject = request.form['subject']

        new_createclass = Createclasses(ccode=ccode, uname=uname, year=year, day=day,
time_start=time_start, time_end=time_end, subject=subject)
        db.session.add(new_createclass)
        db.session.commit()
        return redirect(url_for('classes'))
    return render_template('staff/createclass.html', courses=courses, units=units)
#### Staff class side
@app.route('/classes/', methods=['GET', 'POST'])
def classes():
    cclasses = Createclasses.query.all()
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in

    return render_template('staff/classes.html', cclasses=cclasses)
#### Staff selectioning side
@app.route('/unitselection/', methods=['GET', 'POST'])
def unitselection():
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    else:
        units = Units.query.all()
        courses = Courses.query.all()
    return render_template('staff/unitselection.html' ,units=units,courses=courses)

```

```

@app.route('/search', methods=['post'])
def search():
    if 'eNumber' not in session:
        return redirect('/slogin/')
    keyword = request.form['keyword']
    results = Courses.query.filter(Courses.all.contains(keyword)).all()

    return render_template('staff/unitselection.html', results=results)
##### Staff module Ends
#####

##### student module starts
#####
# Route for registration form
@app.route('/studentregister/', methods=('GET', 'POST'))
def studentregister():
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        regNumber = request.form['regNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        course_id = request.form['course_id']
        passwd1 = request.form['passwd']
        profile_picture = request.files['profile_picture']

        # Save profile picture to server
        filename = secure_filename(regNumber + '_' + profile_picture.filename) # Append
regNumber to original filename
        profile_picture.save('static/uploads/' + filename)

        student = Students(firstName=firstName,
                             lastName=lastName,
                             regNumber=regNumber,
                             email=email,
                             phone=phone,
                             school=school,
                             course_id=course_id,
                             profile_picture=filename,
                             passwd=generate_password_hash(passwd1, method='sha256'))
        db.session.add(student)
        db.session.commit()
        return redirect('/studentlogin/') # Redirect to login page after successful registration
    return render_template('register.html')
# Route for login form

```

```

@app.route('/studentlogin/', methods=('GET', 'POST'))
def studentlogin():
    if request.method == 'POST':
        # Get student input from form
        regNumber = request.form['regNumber']
        passwd = request.form['passwd']
        # Perform login validation, e.g. check if regNumber and password match
        student = Students.query.filter_by(regNumber=regNumber).first()
        if student:
            if check_password_hash(student.passwd, passwd):
                # If login is successful, store regNumber in session
                session['regNumber'] = regNumber
                flash('Login successful!', category='success')
                return redirect('/studentpage/') # Redirect to index page after successful login
            else:
                return render_template('login.html', error='Invalid Registration Number or password')
        return render_template('login.html')
# student index page
@app.route('/studentpage/')
def studentpage():
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in
    else:
        students = Students.query.all()
        return render_template('home.html', students=students)
# student attendance page
@app.route('/studentsattendance/')
def studentsattendance():
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in
    else:
        return render_template('studentattendance.html')
# student classes page
@app.route('/studentclasses/')
def studentclasses():
    cclasses = Createclasses.query.all()
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in

    return render_template('studentclasses.html', cclasses=cclasses)

@app.route('/studentmarkattendance/')

```

```

def takeattendance():
    def resize(img, size):
        width = int(img.shape[1]*size)
        height = int(img.shape[0]*size)
        dimension = (width, height)
        return cv2.resize(img, dimension, interpolation=cv2.INTER_AREA)

    path = "static/uploads"
    studentImg = []
    studentName = []
    myList = os.listdir(path)
    print(myList)

    for cl in myList :
        curImg = cv2.imread(f'{path}/{cl}') # loops in the path student_images/. from the first
        image to the last
        studentImg.append(curImg) #it reads all images saved in the student_images and adds it to
        the studentImg list
        studentName.append(os.path.splitext(cl)[0]) #it will splite the name the image is saved with
        from the extension ".jpg" and saves the name to studentName without extension
        print(studentName)

    def findEncoding(images) : #function to encode the emages in the student_images dir
        imgEncodings = [] # create a list to store all our encodings
        for img in images : # Loop for getting encodings from each image
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            encodeimg = face_rec.face_encodings(img)[0] #add 0 if its an image not a video
            imgEncodings.append(encodeimg)
        return imgEncodings

    EncodeList = findEncoding(studentImg) #find the encodings of student image which are all
    the images in studentsimg that we retrived from the values in the for Loop


def MarkAttendance(name):
    with open(f'attendance/attendance.csv', 'r+') as f: # read and write the file the name of
    student and the time they entered th class
        myDataList = f.readlines()
        nameList = []
        for line in myDataList :
            entry = line.split(',')
            nameList.append(entry[0])

        if name not in nameList :
            now = datetime.now()
            timestr = now.strftime('%H : %M')

```

```

        f.writelines(f'\n{name}, {timestr}')
        statment = str('Welcome to class' + name)

#getting video from webcam
vid = cv2.VideoCapture(0)

while True :
    # get each frame individually
    success, frame = vid.read()
    # resize the frames
    smaller_frames = cv2.resize(frame, (0,0), None, 0.25, 0.25)
    # convert color
    smaller_frames = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    # detect all the faces in one frame
    facesInFrame = face_rec.face_locations(smaller_frames)
    # encode all faces in the frame
    encodeFacesInFrame = face_rec.face_encodings(smaller_frames, facesInFrame)

    #loop that allows to compare all the encodings of faces in the frames with encodings of our
    saved pictures
    for encodeFace, faceloc in zip(encodeFacesInFrame, facesInFrame) : # we are using zip
        because we are comparing two files in one loop
            matches = face_rec.compare_faces(EncodeList, encodeFace)
            facedis = face_rec.face_distance(EncodeList, encodeFace)
            #print(facedis)
            matchIndex = np.argmin(facedis)

            if matches[matchIndex] :
                name = studentName[matchIndex].upper()
                y1, x2, y2, x1 = faceloc
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 3)
                cv2.rectangle(frame, (x1, y2-25), (x2, y2), (0, 255, 0), cv2.FILLED)
                cv2.putText(frame, name, (x1+6, y2-6), cv2.FONT_HERSHEY_COMPLEX, 1, (255,
255, 255), 2)
                MarkAttendance(name)

            #print(name)

    cv2.imshow('video', frame)
    if cv2.waitKey(1) & 0xFF == ord('q') :
        break

    cap.release()
    cv2.destroyAllWindows()

```

```

return render_template('studentclasses.html')

# Log out route
@app.route('/logout/', methods=['POST'])
def logout():
    # Clear the session
    session.clear()
    flash('You have been logged out.', 'success')
    # Redirect to login page after logging out
    return redirect('/')
##### Student module ends
#####

if __name__ == '__main__':
    app.run(debug=True)##### Importing Libraries
#####
from flask import Flask, request, session, redirect, url_for, render_template, flash
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import func
from flask_login import UserMixin, login_manager, login_required, login_user, current_user,
logout_user
from werkzeug.security import generate_password_hash, check_password_hash
import os
from werkzeug.utils import secure_filename
import cv2
from datetime import datetime
import numpy as np
import face_recognition as face_rec

app = Flask(__name__)

basedir = os.path.abspath(os.path.dirname(__file__))

# Configuring database
db = SQLAlchemy()
app.config['SECRET_KEY'] = 'Nokeyisrequired'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database3.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db.init_app(app)

```

```
# Creating Staff Table in Database
```

```
class Staffs(db.Model, UserMixin):
    id = db.Column('staff_id', db.Integer, primary_key=True)
    firstName = db.Column(db.String(100), nullable=False)
    lastName = db.Column(db.String(100), nullable=False)
    eNumber = db.Column(db.String(100), unique=True, nullable=False)
    email = db.Column(db.String(80), unique=True, nullable=False)
    phone = db.Column(db.String(100), unique=True, nullable=False)
    school = db.Column(db.String(100), nullable=False)
    passwd = db.Column(db.String(100), nullable=False)
    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now())

    def __repr__(self):
        return f'<Staff {self.firstName}>'
```

```
# Creating Student Table in Database
```

```
class Students(db.Model, UserMixin):
    id = db.Column(db.Integer, primary_key=True)
    firstName = db.Column(db.String(100), nullable=False)
    lastName = db.Column(db.String(100), nullable=False)
    regNumber = db.Column(db.String(100), unique=True, nullable=False)
    email = db.Column(db.String(80), unique=True, nullable=False)
    phone = db.Column(db.String(100), unique=True, nullable=False)
    school = db.Column(db.String(100), nullable=False)
    course_id = db.Column(db.Integer, db.ForeignKey('courses.id'), nullable=False)
    profile_picture = db.Column(db.String(120), nullable=False)
    passwd = db.Column(db.String(80), nullable=False)
    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now())

    def __repr__(self):
        return f'<Student {self.firstName}>'
```

```
# Creating Courses Table in Database
```

```
class Courses(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    school = db.Column(db.String(100), nullable=False)
    ccode = db.Column(db.String(100), unique=True, nullable=False)
    cname = db.Column(db.String(100), nullable=False)
    units = db.relationship('Units', backref='course', lazy=True)

    def __repr__(self):
        return f'<Course {self.cname}>'
```

```
# Creating Units Table in Database
```

```
class Units(db.Model):
```



```

id = db.Column(db.Integer, primary_key=True)
ccode = db.Column(db.String(100), db.ForeignKey('courses.ccode'), nullable=False)
ucode = db.Column(db.String(100), unique=True, nullable=False)
uname = db.Column(db.String(100), nullable=False)
year = db.Column(db.String(100), nullable=False)

def __repr__(self):
    return f'<Unit {self.unit}>'

##### Creating Staff Table for unite selection in Database
#####
class Unitselection(db.Model):
    id = db.Column('unitselection_id', db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey(Units.ccode), nullable=False)
    ucode = db.Column(db.String(100), db.ForeignKey(Units.ucode), nullable=False)
    uname = db.Column(db.String(100), nullable=False)

    def __repr__(self):
        return f'<Unitselection {self.unitselection_id}>'

##### Creating Staff Table for creating class in Database
#####
class Createclasses(db.Model):
    id = db.Column('createclass_id', db.Integer, primary_key=True)
    ccode = db.Column(db.String(100), db.ForeignKey(Unitselection.ccode), nullable=False)
    uname = db.Column(db.String(100), db.ForeignKey(Unitselection.uname), nullable=False)
    year = db.Column(db.Integer, db.ForeignKey(Units.year), nullable=False)
    day = db.Column(db.VARCHAR(100), nullable=False)
    time_start = db.Column(db.Text)
    time_end = db.Column(db.Text)
    subject = db.Column(db.Text)

    def __repr__(self):
        return f'<Createclasses {self.createclass_id}>'

''' # Inserting data into Courses and Units tables
with app.app_context():
    db.create_all()

#Inserting data into uniselection table
    uniselection1 = Unitselection(ccode='I231',ucode='ITB_5205', uname='MULTMIDIA
TECHNOLOGY')
    uniselection2 = Unitselection(ccode='I321', ucode='IBT_0304', uname='SYSTEM
DEVELOPMENT')
    uniselection3 = Unitselection(ccode='I132', ucode='ITB_2506', uname='MOBILE
COMMUNICATION')

```

```

uniselection4 = Unitselection(ccode='ED', ucode='EDT_4307', uname='ENGILISHI')
uniselection5 = Unitselection(ccode='ED', ucode='ECB_6306', uname='HUMAN
BEHAVIOR')
db.session.add_all([uniselection1, uniselection2, uniselection3, uniselection4, uniselection5])
db.session.commit()

```

```

#Inserting data into uniselection table
class1 = Createclasses(ccode='I231',uname='SYSTEM
DEVELOPMENT',year='2.2',day='monday',time_start='10:00', time_end='12:00')
class2 = Createclasses(ccode='I321',uname='IT
SECURITY',year='4.2',day='friday',time_start='10:00', time_end='12:00')
class3 = Createclasses(ccode='EDS',uname='HUMAN
BEHAVIOR',year='3.2',day='tuesday',time_start='12:00', time_end='14:00')
class4 =
Createclasses(ccode='ED',uname='ENGILISHI',year='1.2',day='monday',time_start='13:00',
time_end='15:00')

```

```

db.session.add_all([class1, class2, class3, class4])
db.session.commit()

```

```

# Inserting data into Courses table
course1 = Courses(school='SIIS', ccode='I231', cname='ICT')
course2 = Courses(school='SIIS', ccode='I321', cname='BIS')
course3 = Courses(school='SIIS', ccode='I132', cname='FCS')
course4 = Courses(school='EDUCATION', ccode='EDS', cname='EDUCATION SCIENCE')
course5 = Courses(school='EDUCATION', ccode='ED', cname='EDUCATION ARTS')
db.session.add_all([course1, course2, course3, course4, course5])
db.session.commit()

```

```

# Inserting data into Units table
unit1 = Units(ccode='I231', ucode='ITB_5205', uname='MULTMIDIA TECHNOLOGY',
year='4.1')
unit2 = Units(ccode='I231', ucode='ITB_1203', uname='PROGRAMMING IN NET',
year='4.1')
unit3 = Units(ccode='I231', ucode='ITB_7204', uname='ADVANCED NETWORKIN',
year='4.1')

unit4 = Units(ccode='I321', ucode='IBT_0304', uname='SYSTEM DEVELOPMENT',
year='4.1')
unit5 = Units(ccode='I321', ucode='IBT_9305', uname='SYSTEM DESIGN', year='4.1')
unit6 = Units(ccode='I321', ucode='IBT_1306', uname='COMPUTER SYSTEM', year='4.1')

unit7 = Units(ccode='I132', ucode='ITB_2506', uname='MOBILE COMMUNICATION',
year='4.1')

```

```

unit8 = Units(ccode='I132', ucode='ITB_1507', uname='OPERATING SYSTEM', year='4.1')
unit9 = Units(ccode='I132', ucode='ITB_8508', uname='IT SECURITY', year='4.1')

unit10 = Units(ccode='EDS', ucode='EIT_3306', uname='KISWAHILI', year='4.1')
unit11 = Units(ccode='ED', ucode='EDT_4307', uname='ENGILISHI', year='4.1')
unit12 = Units(ccode='EDS', ucode='EIT_5308', uname='MULTIMEDIA', year='4.1')
unit13 = Units(ccode='ED', ucode='ECB_6306', uname='HUMAN BEHAVIOR', year='4.1')

db.session.add_all([unit1, unit2, unit3, unit4, unit5, unit6, unit7, unit8, unit9, unit10, unit11,
unit12, unit13])
db.session.commit()

#hassing the password
password = generate_password_hash('password') # Hash the passwords
# Inserting data into Students table
student1 = Students(firstName='John', lastName='Doe', regNumber='12345',
email='john.doe@example.com', phone='1234567890', school='SIIS', course_id=1,
profile_picture='profile.jpg', passwd=password) # Use hashed passwords
student2 = Students(firstName='Jane', lastName='Doe', regNumber='67890',
email='jane.doe@example.com', phone='9876543210', school='SIIS', course_id=2,
profile_picture='profile.jpg', passwd=password)
student3 = Students(firstName='kim', lastName='Doe', regNumber='57697',
email='kim.doe@example.com', phone='7654321098', school='EDUCATION', course_id=3,
profile_picture='profile.jpg', passwd=password)
student4 = Students(firstName='larry', lastName='Doe', regNumber='87993',
email='larry.doe@example.com', phone='0198765432', school='EDUCATION', course_id=4,
profile_picture='profile.jpg', passwd=password)
db.session.add_all([student1,student2,student3, student4])
db.session.commit()

#Inserting data into Staffs table
staff1 = Staffs(firstName='James', lastName='Doe', eNumber='12345',
email='john.doe@example.com', phone='1234567890', school='SIIS', passwd=password) # Use
hashed passwords
staff2 = Staffs(firstName='Jack', lastName='Doe', eNumber='67890',
email='jane.doe@example.com', phone='9876543210', school='SIIS', passwd=password)
staff3 = Staffs(firstName='Johe', lastName='Doe', eNumber='57697',
email='kim.doe@example.com', phone='7654321098', school='EDUCATION',
passwd=password)
staff4 = Staffs(firstName='Dison', lastName='Doe', eNumber='87993',
email='larry.doe@example.com', phone='0198765432', school='EDUCATION',
passwd=password)
db.session.add_all([staff1,staff2,staff3, staff4])
db.session.commit()

#Landing page for the root

```

```

@app.route('/', methods=['GET','POST'])
def index():
    return render_template('index.html')
##### Admin module Starts
#####
## Admin Login Starts
@app.route('/adminlogin/', methods=['GET','POST']) # Admin root
def adminlogin():
    if request.method == 'POST':
        username = request.form['username']
        passwd = request.form['passwd']
        # Perform login validation,
        if username == 'Admin' and passwd == 'admin123':
            # If login is successful, store username in session
            session['username'] = username
            flash('Login successful!', category='success')
            return redirect('/adminhome/') # Redirect to index page after successful login
        else:
            return render_template('admin/login.html', error='username or password is incorrect')
    return render_template('admin/login.html')
## Admin Home Starts
@app.route('/adminhome/')
def adminhome():
    if 'username' not in session:
        return render_template('admin/login.html')
    return render_template('admin/index.html')
## Admin create new student
@app.route('/createstudent/', methods=('GET', 'POST'))
def createstudent():
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        if request.method == 'POST':
            firstName = request.form['firstName']
            lastName = request.form['lastName']
            regNumber = request.form['regNumber']
            email = request.form['email']
            phone = request.form['phone']
            school = request.form['school']
            course_id = request.form['course_id']
            passwd1 = request.form['passwd']
            profile_picture = request.files['profile_picture']

            # Save profile picture to server
            filename = secure_filename(regNumber + '_' + profile_picture.filename) # Append
            regNumber to original filename

```

```

profile_picture.save('static/uploads/' + filename)

student = Students(firstName=firstName,
                    lastName=lastName,
                    regNumber=regNumber,
                    email=email,
                    phone=phone,
                    school=school,
                    course_id=course_id,
                    profile_picture=filename,
                    passwd=generate_password_hash(passwd1, method='sha256'))
db.session.add(student)
db.session.commit()
return redirect(url_for('allstudents'))
return render_template('admin/createstudent.html')
## Admin creat new staff
@app.route('/createstaff/', methods=('GET', 'POST'))
def createstaff():
    if 'username' not in session:
        return render_template('admin/login.html')
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        eNumber = request.form['eNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        passwd1 = request.form['passwd']

        staff = Staffs(firstName=firstName,
                        lastName=lastName,
                        eNumber=eNumber,
                        email=email,
                        phone=phone,
                        school=school,
                        passwd=generate_password_hash(passwd1, method='sha256'))
        db.session.add(staff)
        db.session.commit()
        return redirect(url_for('allstaffs'))
    else:
        return render_template('admin/createstaff.html')
## Admin view all students
@app.route('/allstudents/')
def allstudents():
    if 'username' not in session:
        return render_template('admin/login.html')

```

```

else:
    students = Students.query.all()
    return render_template('admin/students.html', students=students)
## Admin view all staff
@app.route('/allstaffs/')
def allstaffs():
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staffs = Staffs.query.all()
        return render_template('admin/staffs.html', staffs=staffs)
## Admin per student view
@app.route('/studentid/<int:student_id>/')
def student(student_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        student = Students.query.get_or_404(student_id)
        return render_template('admin/student.html', student=student)
## Admin per staff view
@app.route('/staffid/<int:staff_id>/')
def staff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        return render_template('admin/staff.html', staff=staff)
## Admin edit student function
@app.route('/studentid/<int:student_id>/edit/', methods=('GET', 'POST'))
def editstudent(student_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        student = Students.query.get_or_404(student_id)
        if request.method == 'POST':
            firstName = request.form['firstName']
            lastName = request.form['lastName']
            regNumber = request.form['regNumber']
            email = request.form['email']
            phone = request.form['phone']
            school = request.form['school']
            passwd1 = request.form['passwd']

            student.firstName = firstName
            student.lastName = lastName
            student.regNumber = regNumber

```

```

        student.email = email
        student.phone = phone
        student.school = school
        student.passwd = passwd1

        db.session.add(student)
        db.session.commit()
        return redirect(url_for('allstudents'))
    else:
        return render_template('admin/edit.html', student=student)
## Admin edit staff function
@app.route('/staffid/<int:staff_id>/editstaff/', methods=('GET', 'POST'))
def editstaff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        if request.method == 'POST':
            firstName = request.form['firstName']
            lastName = request.form['lastName']
            eNumber = request.form['eNumber']
            email = request.form['email']
            phone = request.form['phone']
            school = request.form['school']
            passwd1 = request.form['passwd']

            staff.firstName = firstName
            staff.lastName = lastName
            staff.eNumber = eNumber
            staff.email = email
            staff.phone = phone
            staff.school = school
            staff.passwd = passwd1

            db.session.add(staff)
            db.session.commit()
            return redirect(url_for('staffs'))
        else:
            return render_template('admin/editstaff.html', staff=staff)
## Admin delete student function
@app.post('/studentid/<int:student_id>/delete/')
def deletestudent(student_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:

```

```

        student = Students.query.get_or_404(student_id)
        db.session.delete(student)
        db.session.commit()
    return redirect(url_for('students'))
## Admin delete Staff function
@app.post('/staffid/<int:staff_id>/delete/')
def deletestaff(staff_id):
    if 'username' not in session:
        return render_template('admin/login.html')
    else:
        staff = Staffs.query.get_or_404(staff_id)
        db.session.delete(staff)
        db.session.commit()
    return redirect(url_for('staffs'))

##### Admin courses directory Starts
#####
@app.route('/courses/')
def courses():
    if 'username' not in session:
        return render_template('admin/login.html')
    courses = Courses.query.all()
    return render_template('admin/courses.html', courses=courses)

##### Admin units directory Starts
#####
@app.route('/units/')
def units():
    if 'username' not in session:
        return render_template('admin/login.html')
    units = Units.query.all()
    return render_template('admin/units.html', units=units)

##### Admin add course directory Starts
#####
@app.route('/createcourse/', methods=('GET', 'POST'))
def createcourse():
    if 'username' not in session:
        return render_template('admin/login.html')
    if request.method == 'POST':
        ccode = request.form['ccode']
        cname = request.form['cname']
        course = Courses(ccode=ccode, cname=cname)

        db.session.add(course)
        db.session.commit()

```



```

        return redirect(url_for('courses'))
    return render_template('admin/createcourse.html')

##### Admin add unit directory Starts
#####
@app.route('/createunit/', methods=('GET', 'POST'))
def createunit():
    if 'username' not in session:
        return render_template('admin/login.html')
    courses = Courses.query.all()
    if request.method == 'POST':
        ccode = request.form['ccode']
        ucode = request.form['ucode']
        uname = request.form['uname']
        year = request.form['year']
        unit = Units(ccode=ccode, ucode=ucode, uname=uname, year=year)

        db.session.add(unit)
        db.session.commit()
        return redirect(url_for('units'))
    return render_template('admin/createunit.html', courses =courses)

##### Admin module Ends
#####

##### Staff module Starts
#####
### Staff register
@app.route('/sregister/', methods=['GET', 'POST'])
def staffregister():
    if request.method == 'POST':
        firstName = request.form['firstName']
        lastName = request.form['lastName']
        eNumber = request.form['eNumber']
        email = request.form['email']
        phone = request.form['phone']
        school = request.form['school']
        passwd = request.form['passwd']

        new_staff = Staffs(firstName=firstName,
                            lastName=lastName,
                            eNumber=eNumber,
                            email=email,
                            phone=phone,
                            school=school,
                            passwd=generate_password_hash(passwd, method='sha256'))

```

```

        db.session.add(new_staff)
        db.session.commit()

        return redirect(url_for('stafflogin'))
    return render_template('staff/register.html')
#### Staff login
@app.route('/slogin/', methods=['GET', 'POST'])
def stafflogin():
    if request.method == 'POST':
        # Get student input from form
        eNumber = request.form['eNumber']
        passwd = request.form['passwd']
        # Perform login validation, e.g. check if regNumber and password match
        staff = Staffs.query.filter_by(eNumber=eNumber).first()
        if staff:
            if check_password_hash(staff.passwd, passwd):
                # If login is successful, store regNumber in session
                session['eNumber'] = eNumber
                flash('Login successful!', category='success')
                return redirect(url_for('staffhome')) # Redirect to index page after successful login
    return render_template('staff/login.html')
#### Staff Homepage
@app.route('/staffHome/')
def staffhome():
    # Check if student is logged in
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    return render_template('staff/home.html')
#### Staff staff attendance side
@app.route('/studentattendance/')
def studentattendance():
    # Check if student is logged in
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    return render_template('staff/attendance.html')
#### Staff create class side
@app.route('/createclass/', methods=['GET', 'POST'])
def createclass():
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    units = Units.query.all()
    courses = Courses.query.all()

    if request.method == 'POST':
        ccode = request.form['ccode']
        uname = request.form['uname']

```

```

    year = request.form['year']
    day = request.form['day']
    time_start = request.form['time_start']
    time_end = request.form['time_end']
    subject = request.form['subject']

    new_createclass = Createclasses(ccode=ccode, uname=uname, year=year, day=day,
time_start=time_start, time_end=time_end, subject=subject)
    db.session.add(new_createclass)
    db.session.commit()
    return redirect(url_for('classes'))
    return render_template('staff/createclass.html', courses=courses, units=units)
#### Staff class side
@app.route('/classes/', methods=['GET', 'POST'])
def classes():
    cclasses = Createclasses.query.all()
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in

    return render_template('staff/classes.html', cclasses=cclasses)
#### Staff selectioning side
@app.route('/unitselection/', methods=['GET', 'POST'])
def unitselection():
    if 'eNumber' not in session:
        return redirect('/slogin/') # Redirect to login page if student is not logged in
    else:
        units = Units.query.all()
        courses = Courses.query.all()
        return render_template('staff/unitselection.html' ,units=units,courses=courses)

@app.route('/search', methods=['post'])
def search():
    if 'eNumber' not in session:
        return redirect('/slogin/')
    keyword = request.form['keyword']
    results = Courses.query.filter(Courses.all.contains(keyword)).all()

    return render_template('staff/unitselection.html', results=results)
##### Staff module Ends
#####

##### student module starts
#####
# Route for registration form
@app.route('/studentregister/', methods=('GET', 'POST'))
def studentregister():

```

```

if request.method == 'POST':
    firstName = request.form['firstName']
    lastName = request.form['lastName']
    regNumber = request.form['regNumber']
    email = request.form['email']
    phone = request.form['phone']
    school = request.form['school']
    course_id = request.form['course_id']
    passwd1 = request.form['passwd']
    profile_picture = request.files['profile_picture']

    # Save profile picture to server
    filename = secure_filename(regNumber + '_' + profile_picture.filename) # Append
regNumber to original filename
    profile_picture.save('static/uploads/' + filename)

    student = Students(firstName=firstName,
                        lastName=lastName,
                        regNumber=regNumber,
                        email=email,
                        phone=phone,
                        school=school,
                        course_id=course_id,
                        profile_picture=filename,
                        passwd=generate_password_hash(passwd1, method='sha256'))
    db.session.add(student)
    db.session.commit()
    return redirect('/studentlogin/') # Redirect to login page after successful registration
return render_template('register.html')
# Route for login form
@app.route('/studentlogin/', methods=('GET', 'POST'))
def studentlogin():
    if request.method == 'POST':
        # Get student input from form
        regNumber = request.form['regNumber']
        passwd = request.form['passwd']
        # Perform login validation, e.g. check if regNumber and password match
        student = Students.query.filter_by(regNumber=regNumber).first()
        if student:
            if check_password_hash(student.passwd, passwd):
                # If login is successful, store regNumber in session
                session['regNumber'] = regNumber
                flash('Login successful!', category='success')
                return redirect('/studentpage/') # Redirect to index page after successful login
            else:
                return render_template('login.html', error='Invalid Registration Number or password')

```

```

    return render_template('login.html')
# student index page
@app.route('/studentpage/')
def studentpage():
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in
    else:
        students = Students.query.all()
        return render_template('home.html', students=students)
# student attendance page
@app.route('/studentsattendance/')
def studentsattendance():
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in
    else:
        return render_template('studentattendance.html')
# student classes page
@app.route('/studentclasses/')
def studentclasses():
    cclasses = Createclasses.query.all()
    # Check if student is logged in
    if 'regNumber' not in session:
        return redirect('/studentlogin/') # Redirect to login page if student is not logged in

    return render_template('studentclasses.html', cclasses=cclasses)

@app.route('/studentmarkattendance/')
def takeattendance():
    def resize(img, size):
        width = int(img.shape[1]*size)
        height = int(img.shape[0]*size)
        dimension = (width, height)
        return cv2.resize(img, dimension, interpolation=cv2.INTER_AREA)

    path = "static/uploads"
    studentImg = []
    studentName = []
    myList = os.listdir(path)
    print(myList)

    for cl in myList :
        curImg = cv2.imread(f'{path}/{cl}') # loops in the path student_images/. from the first
        image to the last

```

```

        studentImg.append(curImg) #it reads all images saved in the student_images and adds it to
the studentImg list
        studentName.append(os.path.splitext(cl)[0]) #it will splite the name the image is saved with
from the extension".jpg" and saves the name to studentName without extension
        print(studentName)

```

```

def findEncoding(images) : #function to encode the emages in the student_images dir
    imgEncodings = [] # create a list to store all our encodings
    for img in images : # Loop for getting encodings from each image
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        encodeimg = face_rec.face_encodings(img)[0] #add 0 if its an image not a video
        imgEncodings.append(encodeimg)
    return imgEncodings

```

```

EncodeList = findEncoding(studentImg) #find the encodings of student image which are all
the images in studentsimg that we retrived from the values in the for Loop

```

```

def MarkAttendance(name):
    with open('attendance/attendance.csv', 'r+') as f: # read and write the file the name of
student and the time they entered th class
        myDataList = f.readlines()
        nameList = []
        for line in myDataList :
            entry = line.split(',')
            nameList.append(entry[0])

        if name not in nameList :
            now = datetime.now()
            timestr = now.strftime('%H : %M')
            f.writelines(f'\n{name}, {timestr}')
            statment = str('Welcome to class' + name)

```

```

#getting video from webcam
vid = cv2.VideoCapture(0)

```

```

while True :
    # get each frame individualy
    success, frame = vid.read()
    # resize the frames
    smaller_frames = cv2.resize(frame, (0,0), None, 0.25, 0.25)
    # convert color
    smaller_frames = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    # detact all the faces in one frame
    facesInFrame = face_rec.face_locations(smaller_frames)
    # encode all faces in the frame

```

```

encodeFacesInFrame = face_rec.face_encodings(smaller_frames, facesInFrame)

#loop that allows to compare all the encodings of faces in the frames with encodings of our
saved pictures
for encodeFace, faceloc in zip(encodeFacesInFrame, facesInFrame) : # we are using zip
because we are comparing two files in one loop
    matches = face_rec.compare_faces(EncodeList, encodeFace)
    facedis = face_rec.face_distance(EncodeList, encodeFace)
    #print(facedis)
    matchIndex = np.argmin(facedis)

    if matches[matchIndex] :
        name = studentName[matchIndex].upper()
        y1, x2, y2, x1 = faceloc
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 3)
        cv2.rectangle(frame, (x1, y2-25), (x2, y2), (0, 255, 0), cv2.FILLED)
        cv2.putText(frame, name, (x1+6, y2-6), cv2.FONT_HERSHEY_COMPLEX, 1, (255,
255, 255), 2)
        MarkAttendance(name)

        #print(name)

    cv2.imshow('video', frame)
    if cv2.waitKey(1) & 0xFF ==ord('q') :
        break

    cap.release()
    cv2.destroyAllWindows()

return render_template('studentclasses.html')

```

```

# Log out route
@app.route('/logout/', methods=['POST'])
def logout():
    # Clear the session
    session.clear()
    flash('You have been logged out.', 'success')
    # Redirect to login page after logging out
    return redirect('/')
##### Student module ends
#####

```

```
if __name__ == '__main__':  
    app.run(debug=True)
```


Index page for the route

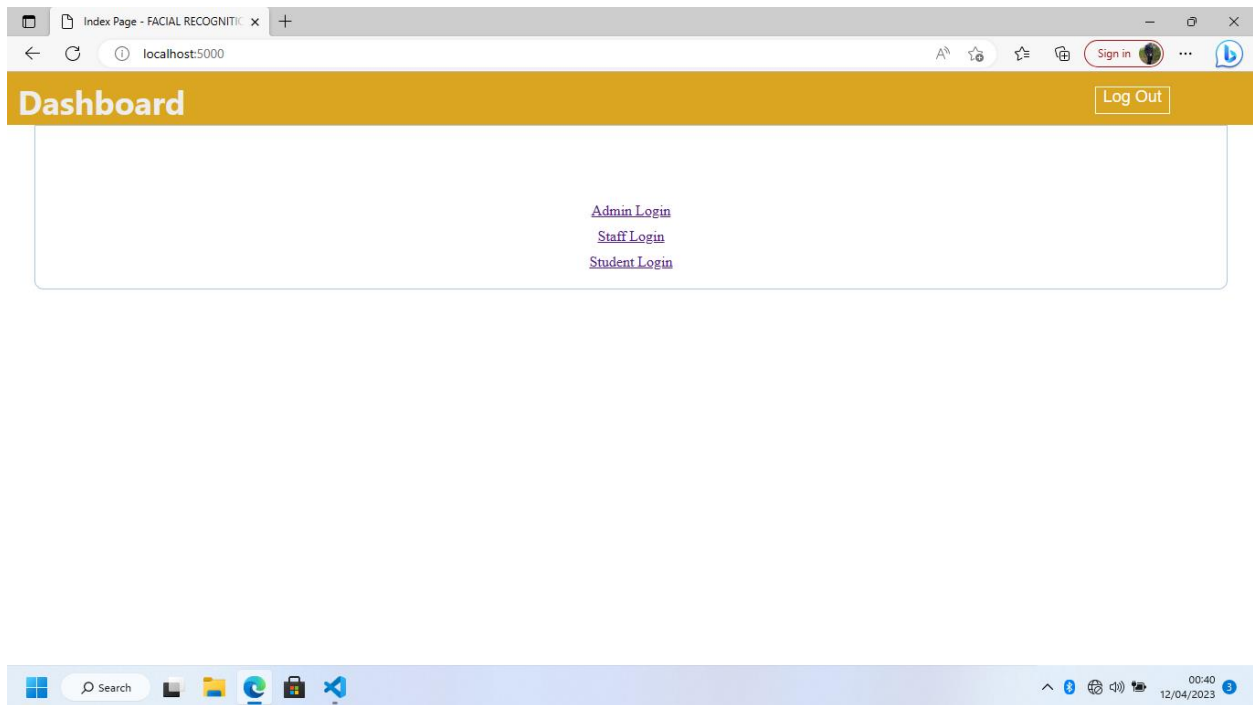
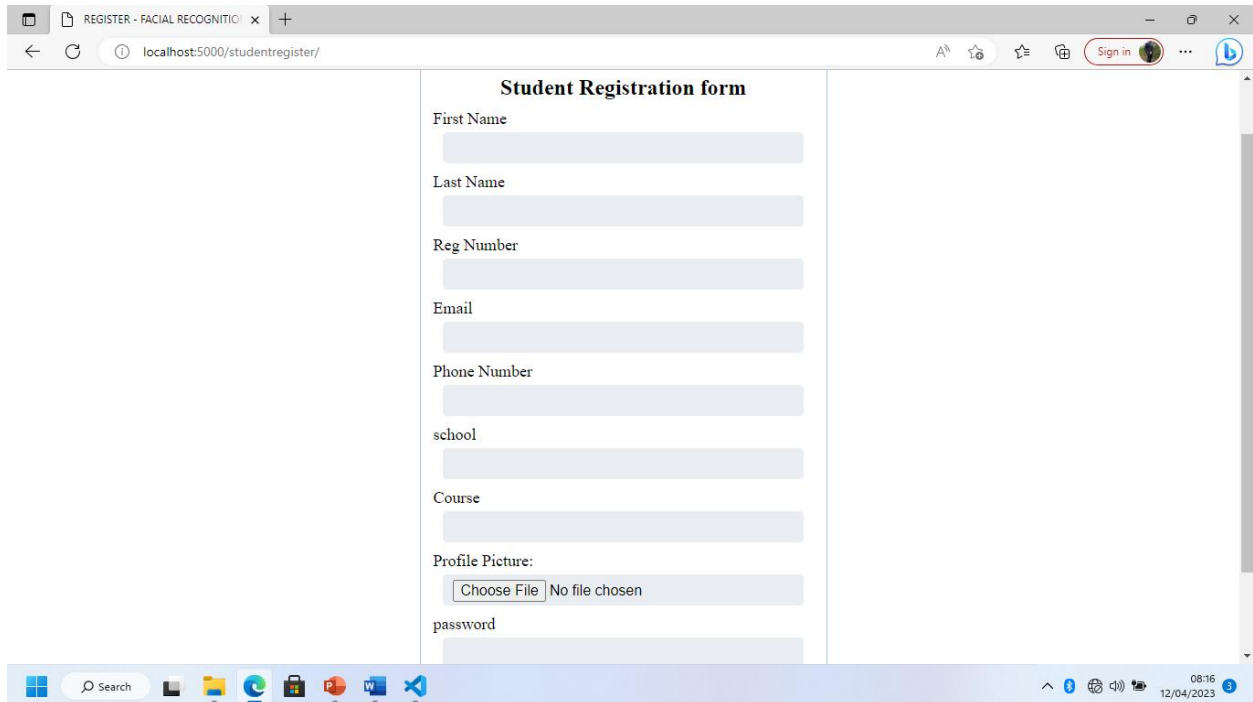


Figure 6- 1 index page for route

Student Module

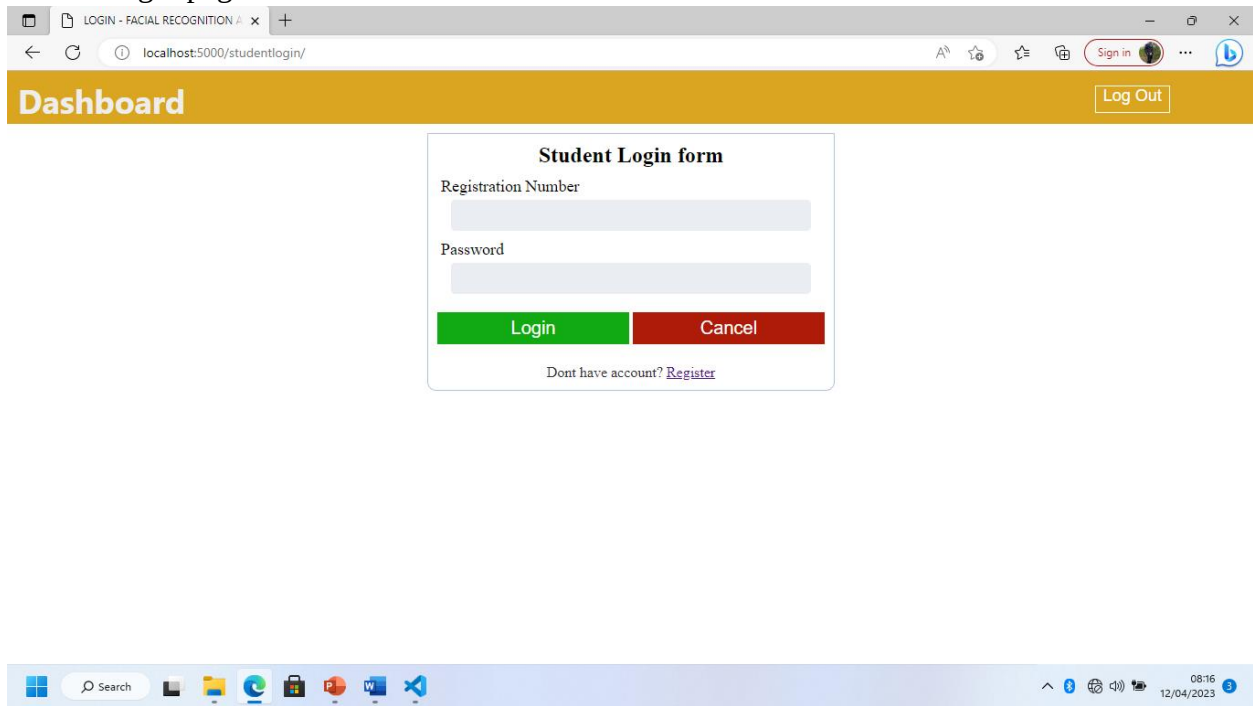
Student registration page



The screenshot shows a web browser window with the address bar displaying "localhost:5000/studentregister/". The page title is "REGISTER - FACIAL RECOGNITION". The main content is a "Student Registration form" with the following fields: First Name, Last Name, Reg Number, Email, Phone Number, school, Course, Profile Picture (with a "Choose File" button and "No file chosen" text), and password. The Windows taskbar at the bottom shows the time as 08:16 on 12/04/2023.

Figure 6- 2 student registration page

Student login page



The screenshot shows a web browser window with the address bar displaying "localhost:5000/studentlogin/". The page title is "LOGIN - FACIAL RECOGNITION". The page has a yellow header bar with the word "Dashboard" on the left and a "Log Out" button on the right. The main content is a "Student Login form" with the following fields: Registration Number and Password. Below the fields are two buttons: "Login" (green) and "Cancel" (red). At the bottom of the form, there is a link: "Dont have account? [Register](#)". The Windows taskbar at the bottom shows the time as 08:16 on 12/04/2023.

Figure 6- 3 student login page

Student home page

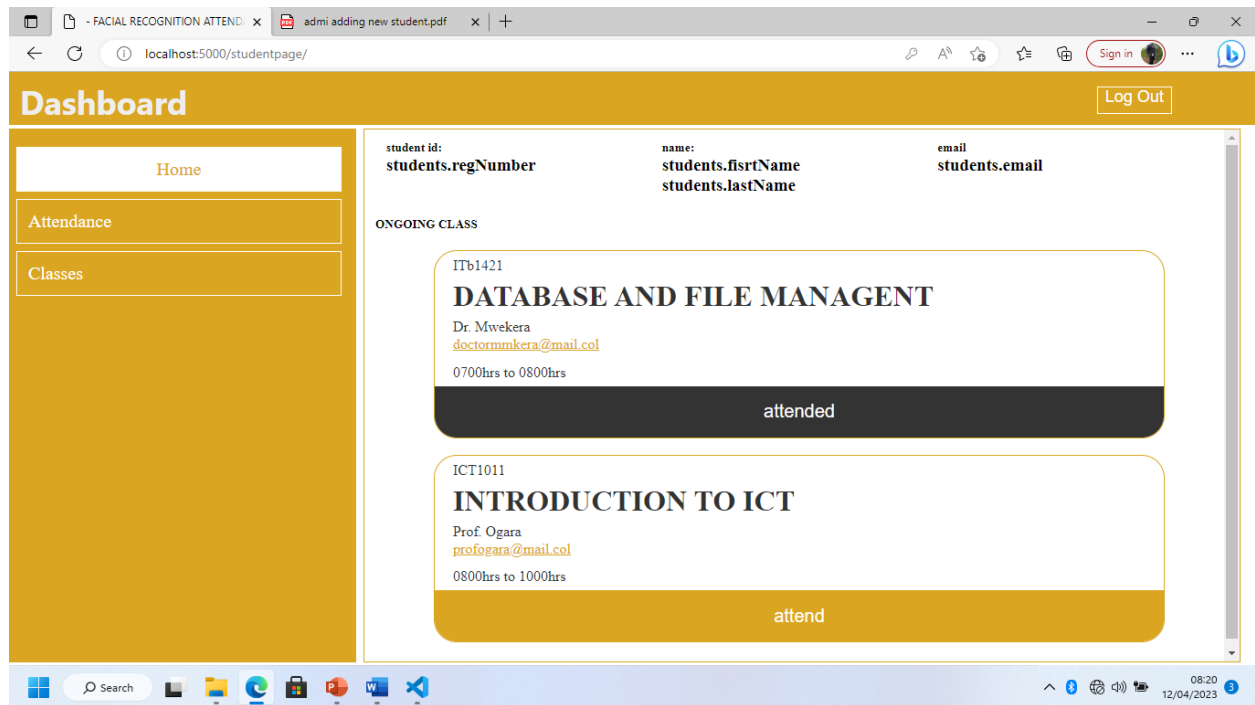


Figure 6- 4 student home page

Student view attendance

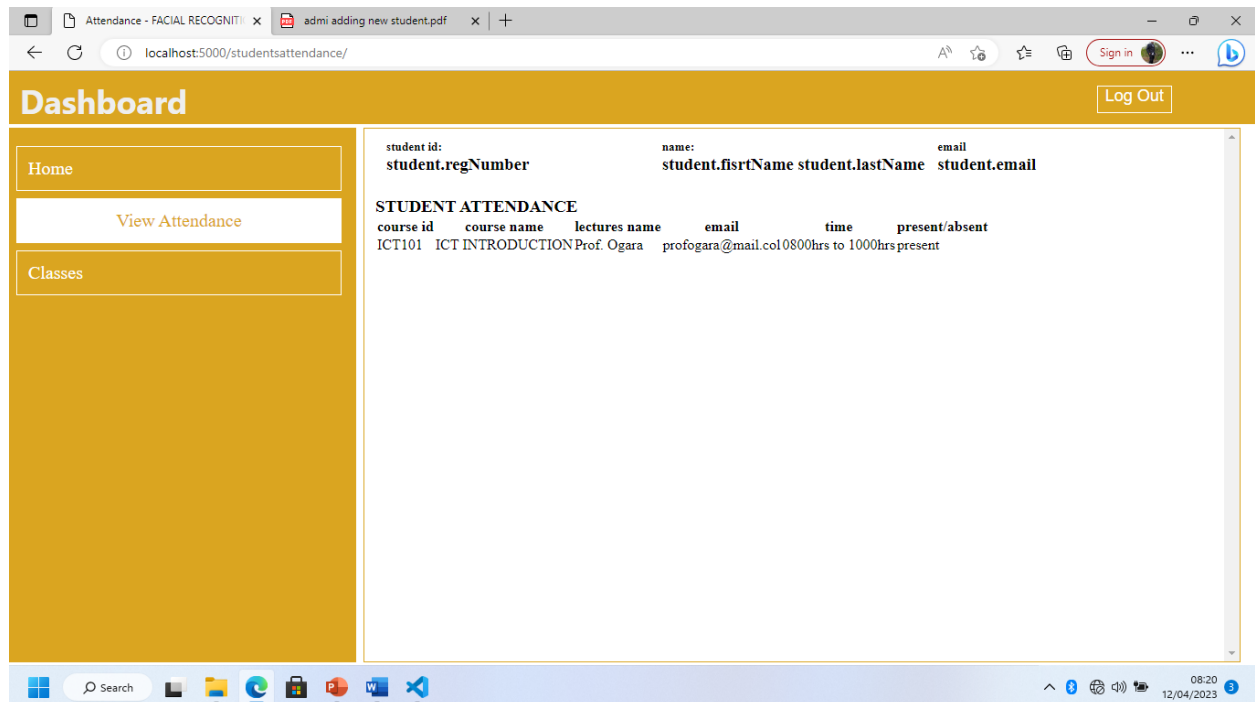


Figure 6- 5 student view attendance

Student viewing and taking attendance per class

The screenshot shows a web browser window with the URL `localhost:5000/studentclasses/`. The page has a yellow header with the word "Dashboard" and a "Log Out" button. On the left, there is a sidebar with three buttons: "Home", "Attendance", and "Classes". The "Classes" button is highlighted. The main content area displays a form for student information with labels: "student id:", "student.regNumber", "name:", "student.fisrtName", "student.lastName", and "email", "student.email". Below this, there is a section titled "ONGOING CLASS" which contains two class cards. The first card is for "ITb1421" titled "DATABASE AND FILE MANAGENT" by "Dr. Mwekera" with email "doctormmkeram@mail.col" and time "0700hrs to 0800hrs". It has a dark grey button labeled "attended". The second card is for "ICT1011" titled "INTRODUCTION TO ICT" by "Prof. Ogara" with email "profogara@mail.col" and time "0800hrs to 1000hrs". It has a yellow button labeled "attend". The Windows taskbar at the bottom shows the time as 08:20 on 12/04/2023.

Dashboard Log Out

Home
Attendance
Classes

student id: **student.regNumber** name: **student.fisrtName student.lastName** email **student.email**

ONGOING CLASS

ITb1421
DATABASE AND FILE MANAGENT
Dr. Mwekera
doctormmkeram@mail.col
0700hrs to 0800hrs
attended

ICT1011
INTRODUCTION TO ICT
Prof. Ogara
profogara@mail.col
0800hrs to 1000hrs
attend

Figure 6- 6 student viewing and attendance taking

Staff Module

Staff registration

The screenshot shows a web browser window with the address bar displaying 'localhost:5000/register/'. The page has a yellow header bar with the word 'Dashboard' on the left and a 'Log Out' button on the right. The main content area contains a 'Staff Registration form' with the following fields: First Name, Last Name, Employee Number, Email, Phone Number, Department, and Password. Each field is represented by a light blue input box. At the bottom of the form are two buttons: 'Register' (blue) and 'cancel' (red). The browser's taskbar at the bottom shows various application icons and the system clock indicating 08:16 on 12/04/2023.

Figure 6- 7 staff registration

Staff login

The screenshot shows a web browser window with the address bar displaying 'localhost:5000/login/'. The page has a yellow header bar with the word 'Dashboard' on the left and a 'Log Out' button on the right. The main content area contains a 'Staff Login form' with the following fields: Employee Number and Password. Each field is represented by a light blue input box. At the bottom of the form are two buttons: 'Login' (green) and 'Cancel' (red). Below the buttons, there is a link that says 'Dont have account? [Register](#)'. The browser's taskbar at the bottom shows various application icons and the system clock indicating 08:16 on 12/04/2023.

Figure 6- 8 staff login

Staff home page

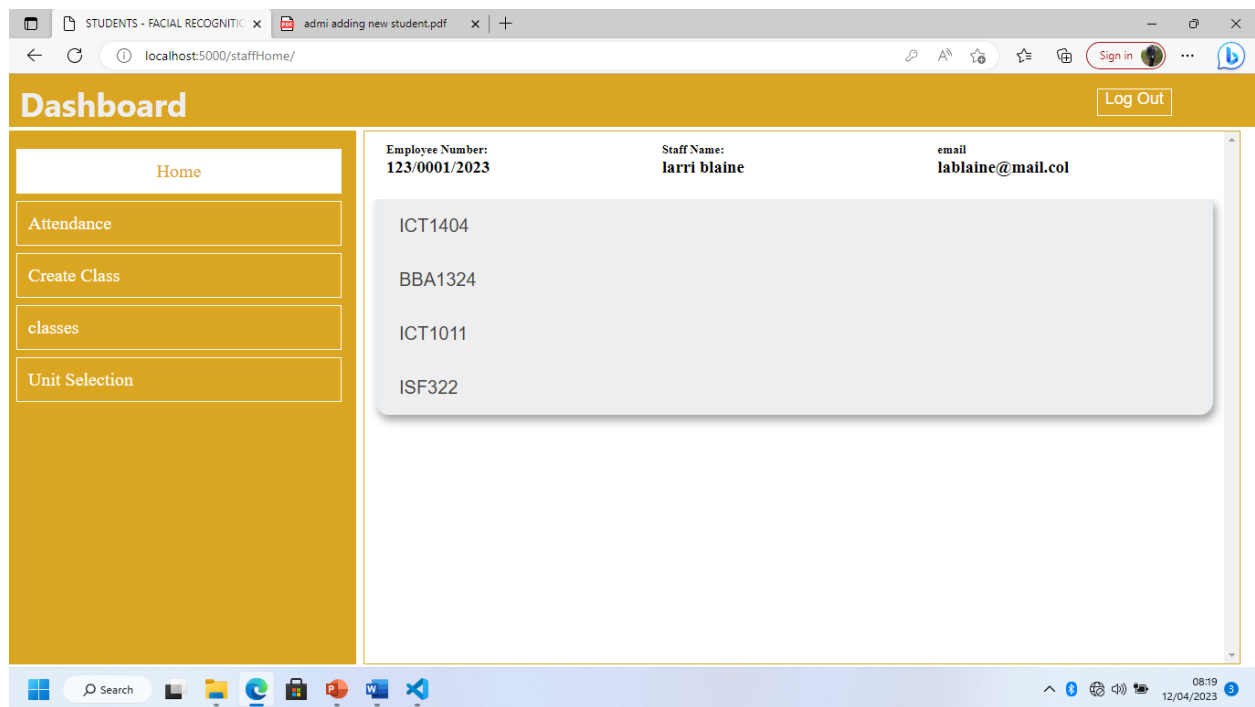


Figure 6- 9 staff home page

Attendance

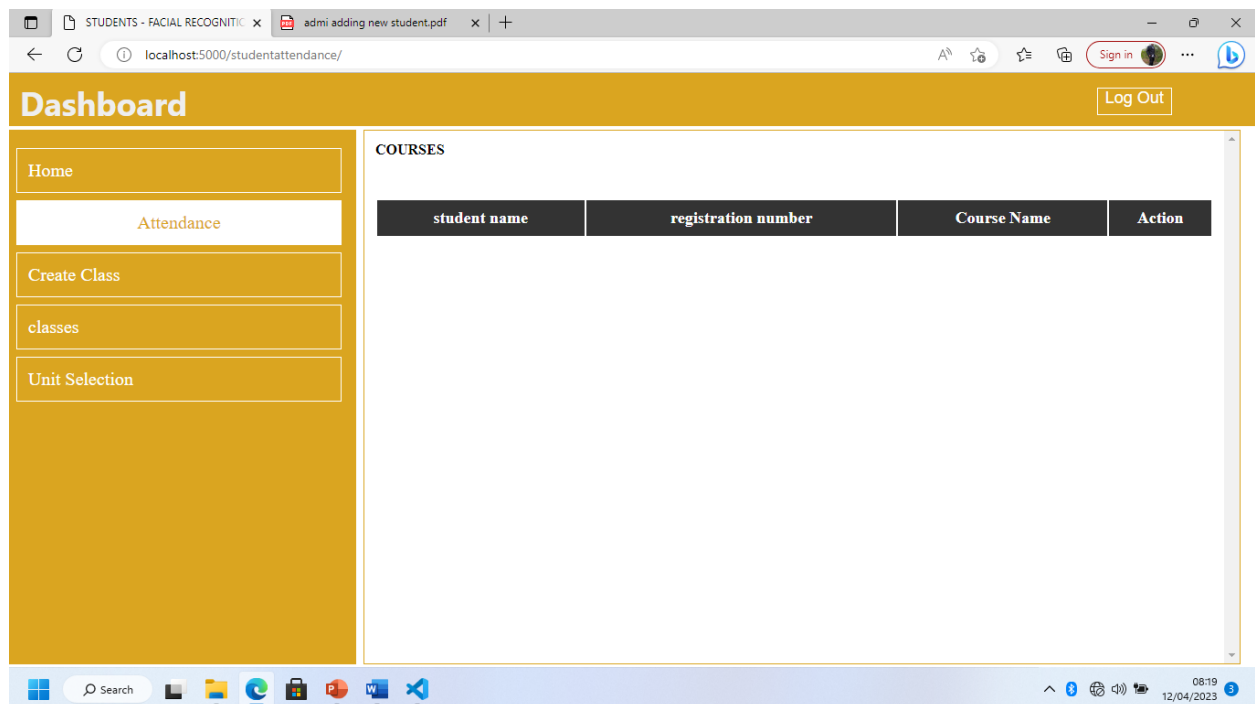


Figure 6- 10 staff attendance

Staff creating classes

STUDENTS - FACIAL RECOGNITIC x admi adding new student.pdf x | +

localhost:5000/createclass/ Sign in Log Out

Dashboard

Home

Attendance

Create Class

classes

Unit Selection

Unit Code: Unit name:
ITB_5205 * MULTIMEDIA TECHNOLOGY

Day
Monday

Time:
From To

Class of Year
One

Subject
Leave a comment here

Create Class

Figure 6- 11 staff creating classes

Staff view courses

STUDENTS - FACIAL RECOGNITIC x admi adding new student.pdf x | +

localhost:5000/classes/ Sign in Log Out

Dashboard

Home

Attendance

Create Class

classes

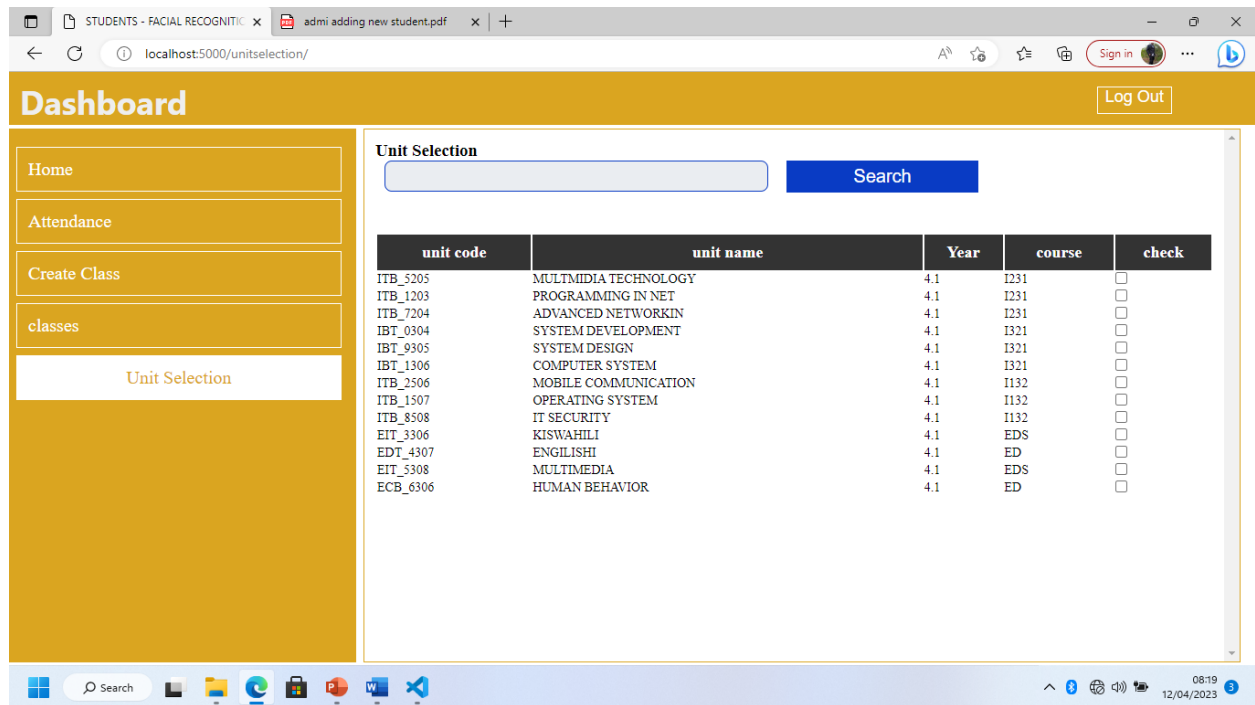
Unit Selection

CLASSES

Course Code	Course Name	Unit Code	Unit Name	Action
-------------	-------------	-----------	-----------	--------

Figure 6- 12 staff view courses

Staff unit selection



The screenshot shows a web application interface for staff unit selection. The browser address bar indicates the URL is `localhost:5000/unitselection/`. The application has a yellow header with a "Dashboard" title and a "Log Out" button. A sidebar on the left contains navigation links: "Home", "Attendance", "Create Class", "classes", and "Unit Selection" (which is highlighted). The main content area is titled "Unit Selection" and features a search bar and a "Search" button. Below these is a table with the following data:

unit code	unit name	Year	course	check
ITB_5205	MULTIMEDIA TECHNOLOGY	4.1	I231	☐
ITB_1203	PROGRAMMING IN NET	4.1	I231	☐
ITB_7204	ADVANCED NETWORKIN	4.1	I231	☐
IBT_0304	SYSTEM DEVELOPMENT	4.1	I321	☐
IBT_9305	SYSTEM DESIGN	4.1	I321	☐
IBT_1306	COMPUTER SYSTEM	4.1	I321	☐
ITB_2506	MOBILE COMMUNICATION	4.1	I132	☐
ITB_1507	OPERATING SYSTEM	4.1	I132	☐
ITB_8508	IT SECURITY	4.1	I132	☐
EIT_3306	KISWAHILI	4.1	EDS	☐
EDT_4307	ENGLISHI	4.1	ED	☐
EIT_3308	MULTIMEDIA	4.1	EDS	☐
ECB_6306	HUMAN BEHAVIOR	4.1	ED	☐

Figure 6- 13 staff unit selection

Admin Module

Admin Login page

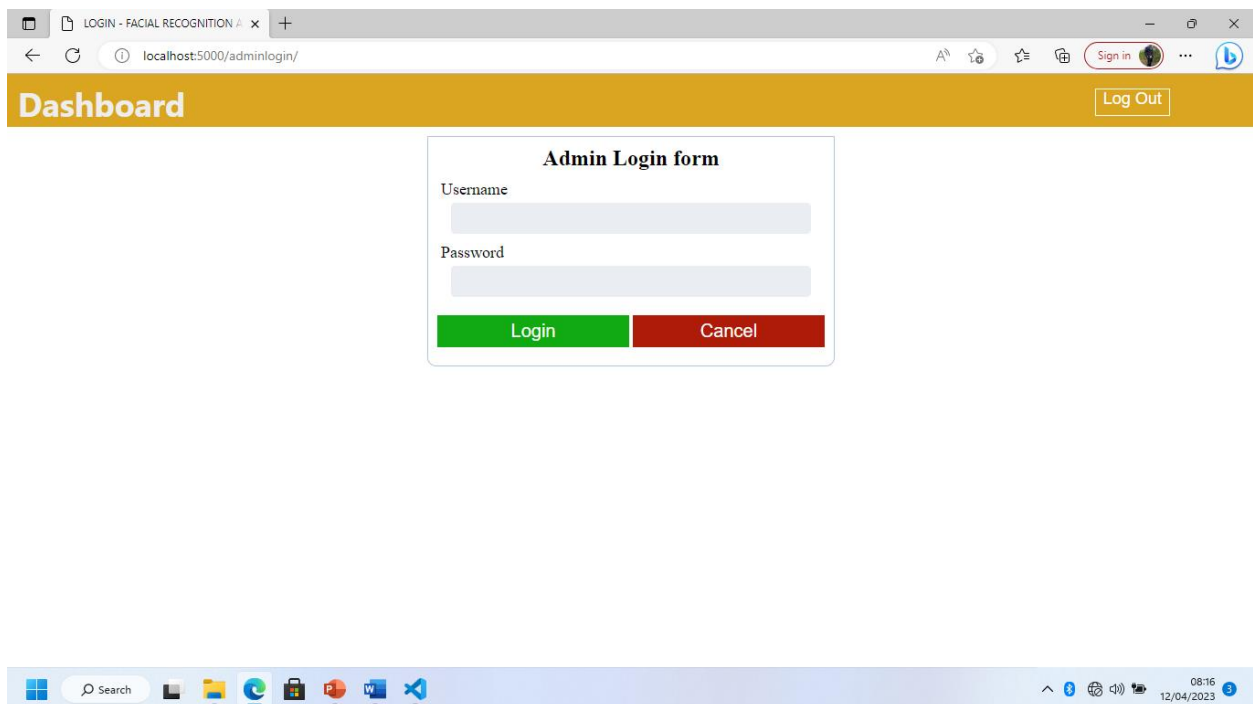


Figure 6- 14 admin login page

Admin Index page

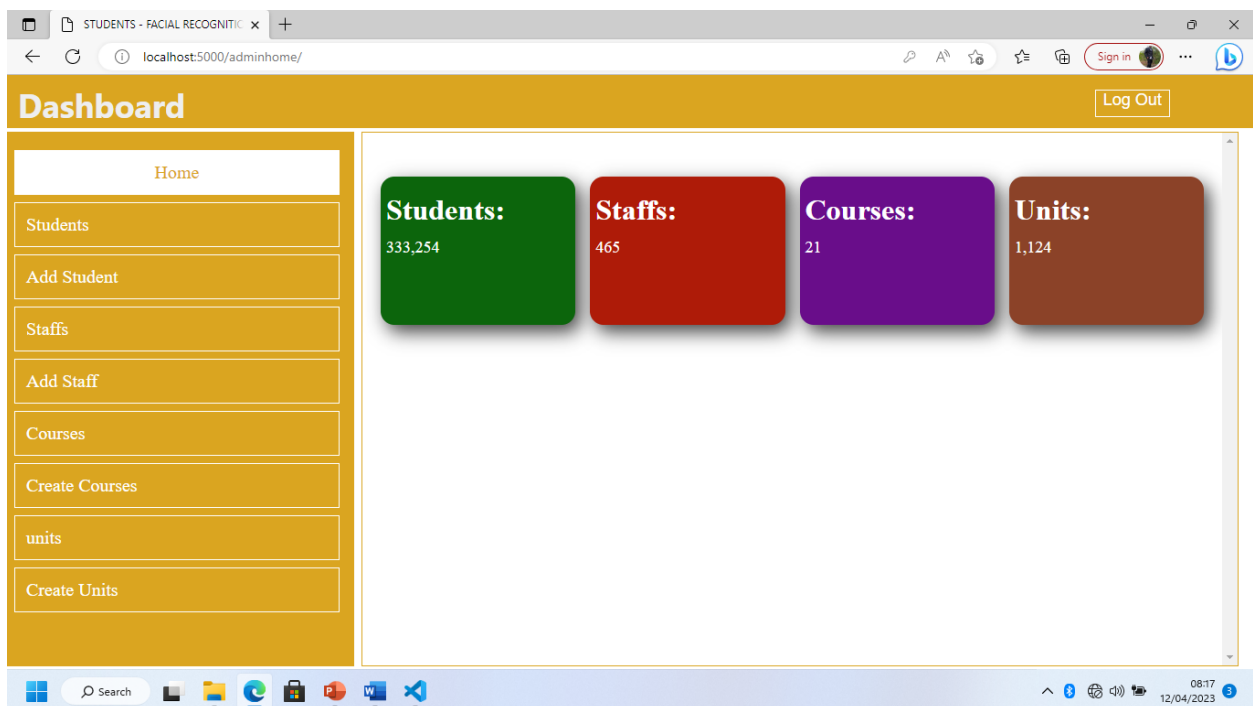


Figure 6- 15 admin home page

Admin view all students

The screenshot shows a web application titled "STUDENTS - FACIAL RECOGNITION" running on localhost:5000/allstudents/. The dashboard has a yellow header with a "Log Out" button. A left sidebar contains navigation links: Home, Students (highlighted), Add Student, Staffs, Add Staff, Courses, Create Courses, units, and Create Units. The main content area, titled "STUDENTS", displays a table with 7 rows of student data. Each row includes an ID, Student id, Name, Email, Phone No, School, and an Action button labeled "View details".

ID	Student id	Name	Email	Phone No	School	Action
1	12345	John Doe	john.doe@example.com	1234567890	SIIS	View details
2	67890	Jane Doe	jane.doe@example.com	9876543210	SIIS	View details
3	57697	kim Doe	kim.doe@example.com	7654321098	EDUCATION	View details
4	87993	larry Doe	larry.doe@example.com	0198765432	EDUCATION	View details
5	i231	Brian Kimonge	kimw@gmail.com	75624656	SIIS	View details
6	i231/0050/2019	Brian Kimonge	kim23@gmail.com	7562465669	SIIS	View details
7	12	kim larry	kimlarry59@gmail.com	7562669	EDUCATION	View details

Figure 6- 16 admin view all students

Admin adding students

The screenshot shows the "Add New Student" form in the same web application, running on localhost:5000/createstudent/. The dashboard header and sidebar are identical to the previous figure. The main content area is titled "Add New Student" and contains a form with the following fields: First Name, Last Name, Reg Number, Email, Phone Number, school, and Course. Each field has a corresponding text input box.

Figure 6- 17 admin add student

Admin view staffs

The screenshot shows a web application interface for viewing staff. The dashboard has a sidebar menu with options: Home, Students, Add Student, Staffs (highlighted), Add Staff, Courses, Create Courses, units, and Create Units. The main content area is titled 'STAFFS' and displays a table with the following data:

ID	Employee Id	Name	Email	Phone No	Department	Action
1	12345	James Doe	john.doe@example.com	1234567890		View details
2	67890	Jack Doe	jane.doe@example.com	9876543210		View details
3	57697	Johe Doe	kim.doe@example.com	7654321098		View details
4	87993	Dison Doe	larry.doe@example.com	0198765432		View details
5	12	kim larry	kimlarry59@gmail.com	7562ti9		View details
6	1	kim larry	karry59@gmail.com	9		View details

Figure 6- 18 admin view staffs

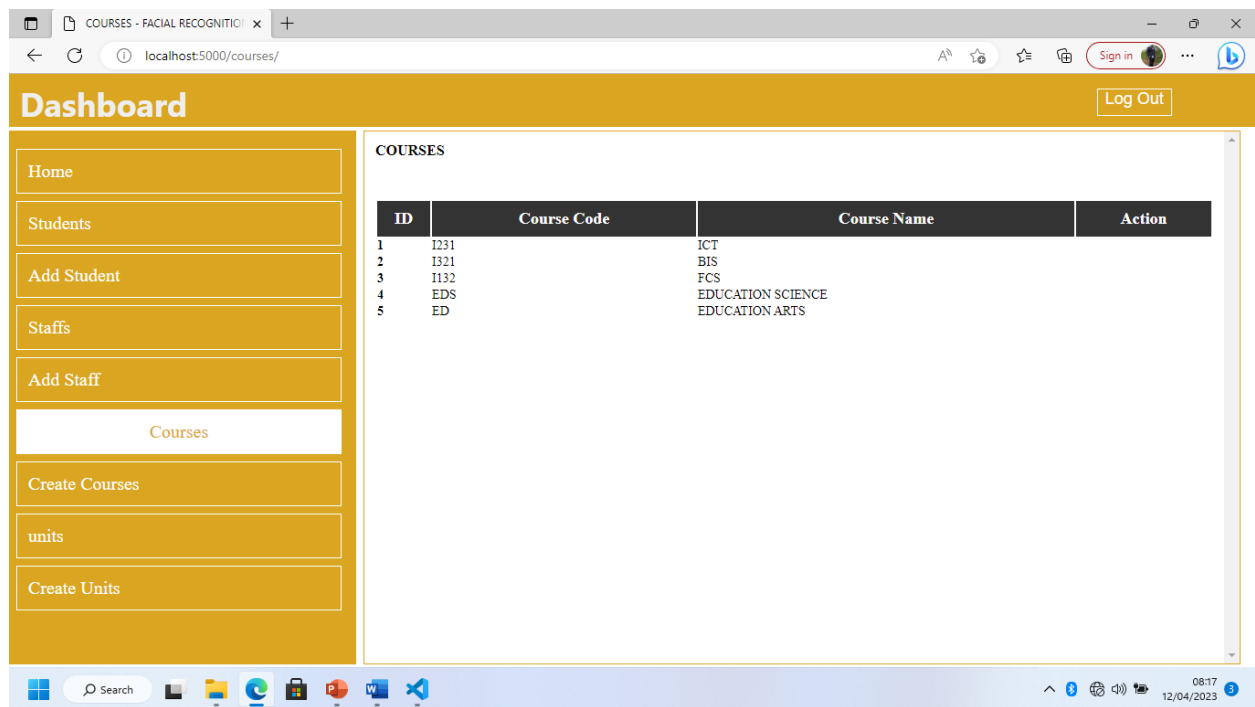
Admin adding staffs

The screenshot shows a web application interface for adding a new staff member. The dashboard has a sidebar menu with options: Home, Students, Add Student, Staffs, Add Staff (highlighted), Courses, Create Courses, units, and Create Units. The main content area is titled 'Add New Staff' and displays a form with the following fields:

- First Name
- Last Name
- Employee Number
- Email
- Phone Number
- School
- password

Figure 6- 19 admin adding staffs

Admin view all courses

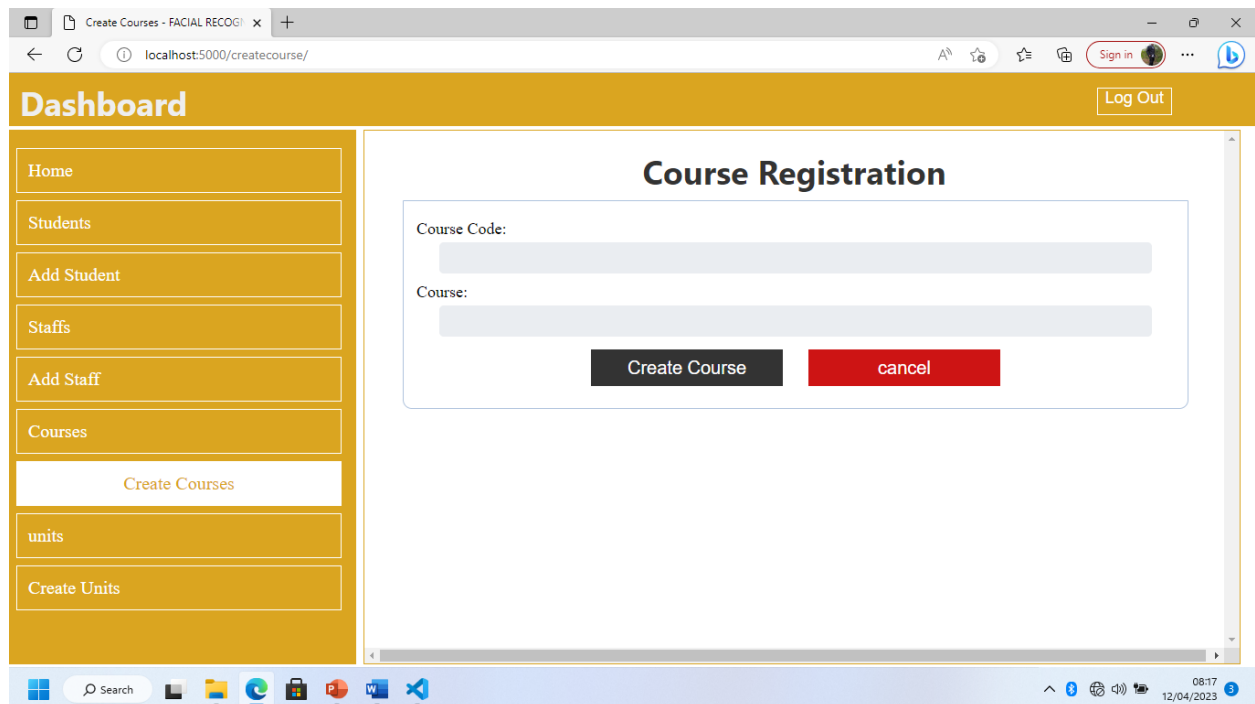


The screenshot shows a web application interface for managing courses. The dashboard has a sidebar menu on the left with options: Home, Students, Add Student, Staffs, Add Staff, Courses (highlighted), Create Courses, units, and Create Units. The main content area is titled "COURSES" and displays a table with the following data:

ID	Course Code	Course Name	Action
1	I231	ICT	
2	I321	BIS	
3	I132	FCS	
4	EDS	EDUCATION SCIENCE	
5	ED	EDUCATION ARTS	

Figure 6- 20 admin view all courses

Admin course registration



The screenshot shows the "Create Courses" page in the web application. The sidebar menu is identical to the previous screenshot, with "Create Courses" highlighted. The main content area is titled "Course Registration" and contains a form with the following fields and buttons:

- Course Code:
- Course:
-
-

Figure 6- 21 admin course registration

Admin view all units

The screenshot shows a web application interface for viewing all units. The dashboard has a sidebar menu with the following items: Home, Students, Add Student, Staffs, Add Staff, Courses, Create Courses, units (highlighted), and Create Units. The main content area is titled "UNITS" and displays a table with the following data:

ID	Course Code	Unit Code	Unit Name	Year of Study	Action
1	I231	ITB_5205	MULTIMEDIA TECHNOLOGY	4.1	
2	I231	ITB_1203	PROGRAMMING IN NET	4.1	
3	I231	ITB_7204	ADVANCED NETWORKIN	4.1	
4	I321	IBT_0304	SYSTEM DEVELOPMENT	4.1	
5	I321	IBT_9305	SYSTEM DESIGN	4.1	
6	I321	IBT_1306	COMPUTER SYSTEM	4.1	
7	I132	ITB_2506	MOBILE COMMUNICATION	4.1	
8	I132	ITB_1507	OPERATING SYSTEM	4.1	
9	I132	ITB_8508	IT SECURITY	4.1	
10	EDS	EIT_3306	KISWAHILI	4.1	
11	ED	EDT_4307	ENGLISHI	4.1	
12	EDS	EIT_5308	MULTIMEDIA	4.1	
13	ED	ECB_6306	HUMAN BEHAVIOR	4.1	

Figure 6- 22 admin view all units

Admin unit registration

The screenshot shows a web application interface for unit registration. The dashboard has a sidebar menu with the following items: Home, Students, Add Student, Staffs, Add Staff, Courses, Create Courses, units (highlighted), and Create Units. The main content area is titled "Course Registration" and displays a form with the following fields:

- Course Code: (dropdown menu showing I231-)
- Unit Code: (text input field)
- Unit: (text input field)
- Year of Study: (text input field)

At the bottom of the form, there are two buttons: "Create Course" (black) and "cancel" (red).

Figure 6- 23 admin unit registration

7. WORK PLAN

PROJECT PHASE	ACTIVITY	RESOURCES	DELIVARABLES	TIME (days)
1.PLANNING	1. Document a project plan 2. Divide tasks among team members 3. Defining project deliverables and requirement 4. Creating project schedule	1. System analyst 2. Team manager Project manager	<u>Project charter.</u> -Objectives -Stakeholders -Problem -statement -Scope	20 days
2.DESIGN	Developing a use case Data flow diagram Context diagram DFD and ERD diagram	1. Project manager 2. System Analyst 3. System designer	Use case diagram DFD diagram Context diagram ERD diagram	30 days
3. IMPLEMETATION AND PLANNING	1. Analyse feasibility of each Coding language. 2. Programming	1. Programmers 2. Analysts 3. Project Manager	Code Code documentation	30 days
4.TESTING	1. Unit Testing 2. Integration testing 3. System testing 4. Debugging	1. Programmer 2. Team members	<u>Status report</u> Test case Defect documentation Test plan	15
5.DEPLOYMENT	1. Installation 2. Performance monitoring	1. Programmers 2. Project Manager	Deployment plan Web based Application system	10
6.MAINTENANCE	1. Updates 2. Repairs 3. Bug fixing	1. Programmer 2. System administrated	Updated version	lifetime

Table 1 : Work Plan

8. BUDGET

ACTIVITIES	QUANTITY	RATE	TOTAL
i. Stationery- Notebooks A4	2 reams	300.00	600.00
-Pens	4	25 x 4	100.00
-Flash disk	4 GB	799	750.00
-Spring files	3 pieces	50.00	150.00
ii. Photocopying	10 copies	200.00	2,000.00
iii. Binding (Loosely)	10	50.00	500.00
iv. Literature review- internet research	30 days (Online)	100.00	3,000.00
v. Laptop	2	28,000 x 2	56,000.00
Subtotal			63,100
GRAND TOTAL			
Ksh. 64,000			

Table 2: Project Budget.

9. REFERENCE

1. History of facial recognition and areas applicability <https://us.norton.com/blog/iot/how-facial-recognition-software-works#>
2. What is facial recognition? How does it work? <https://www.kaspersky.com/resource-center/definitions/what-is-facial-recognition>
3. Schools using facial recognition system sparks privacy concerns in China <https://edtechchina.medium.com/schools-using-facial-recognition-system-sparks-privacy-concerns-in-china-d4f706e5cfd0>
4. Gates, Kelly (2011). Our Biometric Future: Facial Recognition Technology and the Culture of Surveillance. NYU Press. Pg.49-54.
5. Nilsson, Nils J. (2009). The Quest for Artificial Intelligence. Cambridge University Press.

